



Kierunek: Informatyka

2025/2026

Autor:

Maksymilian Augustyn

PRACA INŻYNIERSKA

Interaktywna mapa katastralna Gminy Czarna z XIX wieku

Promotor pracy:

dr inż. Adam Pieprzcki

Tarnów, 2026

Spis treści

Wstęp	4
Rozdział 1. Cel i założenia projektowe.....	5
1.1. Cel i zakres pracy.....	5
1.2. Założenia i ograniczenia projektu.....	6
1.3. Słownik kluczowych pojęć	6
Rozdział 2. Analiza materiałów źródłowych	9
2.1. Dane katastralne: protokoły i mapa	9
2.2. Dane genealogiczne: księgi metrykalne.....	9
Rozdział 3. Elementy budowy systemu i wykorzystane technologie	11
3.1. Architektura systemu	11
3.2. Uzasadnienie wyborów technologicznych.....	13
Rozdział 4: Projekt bazy danych.....	15
4.1. Architektura <i>Multitenancy</i> (Wielodostępność)	15
4.2. Diagram związków encji (ERD)	15
4.3. Szczegółowy opis kluczowych tabel.....	16
4.4. Integralność danych	19
Rozdział 5: Implementacja ekosystemu narzędziowego	20
5.1. Launcher (launcher/launcher_app.py) – zintegrowane centrum dowodzenia	20
5.2. Edytor właścicieli (tools/owner_editor.py).....	23
5.3. Edytor działek (tools/parcel_editor/parcel_editor_app.py).....	27
5.4. Edytor genealogii (tools/genealogy_editor/editor_app.py).....	30
Rozdział 6: Implementacja backendu i procesu migracji	33
6.1. Serwer <i>API</i> (backend/app.py)	33
6.2. Mechanizmy bezpieczeństwa.....	36
6.3. Cykl życia danych i proces migracji.....	37
6.4. Obsługa błędów i kody statusu API.....	37
6.5. Skrypt migracyjny (backend/migrate_data.py).....	39
Rozdział 7: Implementacja aplikacji frontendowej	40
7.1. Mapa interaktywna (mapa/)	40
7.2. Panel administracyjny (admin/)	43
7.3. Centrum analityczne – statystyki i rankingi (wlascciele/stats.html).....	47
7.4. Moduł genealogiczny (genealogia/ i graf/) – wizualizacja sieci społecznych.....	53

7.5. Strona wprowadzająca i materiały uzupełniające	60
Rozdział 8. Testowanie i zapewnienie jakości.....	64
8.1. Cel i zakres testów	64
8.2. Środowisko testowe	65
8.3. Opis przypadków testowych	65
8.4. Uruchamianie testów i dalszy rozwój	66
Rozdział 9. Podsumowanie.....	67
9.1. Osiągnięte rezultaty.....	67
9.2. Wnioski końcowe.....	68
9.3. Propozycje dalszego rozwoju.....	69
Bibliografia	71

Wstęp

Niniejsza dyplomowa praca inżynierska opisuje kompleksowy proces projektowania, implementacji oraz wdrożenia zintegrowanego systemu informatycznego służącego do wizualizacji i analizy historycznych danych katastralnych. **Problem badawczy**, który stanowił punkt wyjścia dla prezentowanego w niej projektu, dotyczy wyzwań związanych z digitalizacją, ustrukturyzowaniem i udostępnianiem danych archiwalnych. Materiały te, w swojej pierwotnej, papierowej formie, są nie tylko trudno dostępne dla szerokiego grona odbiorców, ale również podatne na nieodwracalną degradację fizyczną.

Celem pracy jest zaprezentowanie kompletnego rozwiązania, które transformuje statyczne, analogowe zapisy w dynamiczną, interaktywną bazę wiedzy, dostępną przez nowoczesną aplikację internetową.

Rozdział 1. Cel i założenia projektowe

Niniejszy rozdział przedstawia fundamenty projektowe pracy, definiując cele, zakres oraz kluczowe założenia systemowe. Określono również słownik pojęć domenowych oraz strukturę organizacyjną projektu, co stanowi niezbędną podstawę dla dalszych rozdziałów.

1.1. Cel i zakres pracy

Celem głównym pracy było stworzenie, w formie aplikacji webowej *full-stack* w pełni funkcjonalnej, interaktywnej mapy katastralnej Gminy Czarna, bazującej na danych z 1882 roku. System ten ma za zadanie nie tylko cyfrowo zabezpieczyć cenne dane historyczne, ale również “ożywić” je, udostępniając w przystępnej i dynamicznej formie szerokiemu gronu odbiorców – od historyków i genealogów, po lokalnych pasjonatów i potomków dawnych mieszkańców.

Do realizacji celu głównego zdefiniowano następujące **cele szczegółowe**, które wyznaczyły zakres prac:

- **digitalizacja i integracja zasobów archiwalnych** – przetworzenie danych z analogowych map, protokołów katastralnych do ustrukturyzowanej postaci cyfrowej i zintegrowanie ich w jednej, spójnej bazie danych;
- **projekt i implementacja bazy danych** – utworzenie wydajnego i skalowalnego schematu relacyjnej bazy danych z wykorzystaniem rozszerzenia przestrzennego PostGIS do obsługi geometrii obiektów;
- **budowa ekosystemu narzędziowego** – opracowanie dedykowanych, graficznych aplikacji deweloperskich (*GUI*) w celu zapewnienia integralności i poprawności danych już na etapie ich wprowadzania;
- **implementacja warstwy backendowej** – zbudowanie serwera *API* w technologii Python/Flask, który zarządza logiką biznesową i pełni rolę pośrednika między bazą danych a interfejsem użytkownika;

- **implementacja warstwy frontendowej** – zaprojektowanie i wdrożenie interaktywnego interfejsu użytkownika z wykorzystaniem technologii HTML, CSS, JavaScript oraz biblioteki Leaflet.js do wizualizacji danych kartograficznych;
- **rozwój modułów analitycznych** – wzbogacenie systemu o funkcjonalności wykraczające poza prostą wizualizację, takie jak generowanie statystyk, porównywanie protokołów oraz eksploracja powiązań genealogicznych.

Projekt stanowi również odpowiedź na osobiste zainteresowania autora historią regionu oraz chęć wykorzystania nowoczesnych technologii do ochrony i popularyzacji lokalnego dziedzictwa kulturowego.

1.2. Założenia i ograniczenia projektu

Projekt opiera się na następujących kluczowych założeniach:

- **wierność źródłom** – system musi jak najwierniej odwzorowywać dane zawarte w oryginalnych dokumentach, z uwzględnieniem historycznej terminologii i formy zapisu;
- **otwartość technologiczna** – rozwiązanie bazuje wyłącznie na otwartych i darmowych technologiach (*Open Source*), co minimalizuje koszty wdrożenia i zapewnia możliwość dalszego, nieograniczonego rozwoju;
- **Modularność architektury** – architektura systemu powinna być rozszerzalna, aby w przyszłości umożliwić łatwe dodawanie nowych funkcjonalności (np. obsługa map z innych okresów).

Głównym **ograniczeniem projektu** jest jakość materiałów źródłowych. Nieczytelność niektórych fragmentów mapy oraz ręcznie pisanych protokołów uniemożliwiła pełną digitalizację 100% obiektów i danych. Aplikacja prezentuje tak kompletny zbiór informacji, jak było to możliwe do odtworzenia na podstawie dostępnych archiwaliów.

1.3. Słownik kluczowych pojęć

- **protokół katastralny** – Oficjalny dokument z XIX wieku, stanowiący szczegółowy zapis stanu posiadania dla jednego właściciela (lub grupy współwłaścicieli).

- **własność rzeczywista** – zbiór działek, które na podstawie analizy protokołu i mapy zostały zweryfikowane jako faktyczna własność danej osoby w badanym okresie;
- **własność wg protokołu** – zbiór działek przypisanych do osoby w oryginalnym dokumencie, który mógł zawierać błędy pisarskie lub nieaktualne informacje. Rozróżnienie to pozwala na analizę rozbieżności w danych historycznych;
- **Obiekty specjalne** – kategoria obiektów punktowych o szczególnym znaczeniu dla społeczności, np. dworzec kolejowy, młyn, kościół;
- **Infrastruktura** – kategoria obiektów liniowych, które nie stanowiły własności prywatnej, takich jak drogi publiczne i rzeki.

1.4. Struktura projektu

Projekt posiada modułową strukturę katalogów, co ułatwia nawigację po kodzie i dalszy rozwój aplikacji. Ze względu na wdrożenie obsługi wielu miejscowości, struktura została zorganizowana tak, aby oddzielić kod aplikacji od danych specyficznych dla danej lokalizacji.

Poniżej przedstawiono opis kluczowych katalogów w projekcie:

- **admin/** – kompletna aplikacja panelu administracyjnego (*SPA*) służąca do zarządzania danymi w bazie PostgreSQL;
- **backup/** – główny magazyn danych, w przeciwieństwie do standardowych rozwiązań, folder ten pełni rolę bazy źródłowej dla mechanizmu wielomiejscowości, zawiera podkatalogi dla każdej gminy (np. backup/Czarna/, backup/Pilzno/), w których znajdują się:
 - pliki konfiguracyjne (launcher_db_config.json, map_config.json),
 - zbiory danych JSON (właściciele, działki, genealogia),
 - cyfrowe skany protokołów i mapy historyczne;
- **backend/** – logika serwerowa aplikacji (*API*) napisana w języku Python (Flask) oraz skrypty migracyjne;
- **docs/** – Interaktywna dokumentacja techniczna i przewodnik wdrożeniowy generowany w HTML;
- **launcher/** – Kod źródłowy aplikacji desktopowej „Launcher”, która zarządza uruchamianiem serwera i konfiguracją baz danych;

- mapa/, genealogia/, graf/, właściciele/ – Katalogi zawierające kod frontendowy (HTML/JS/CSS) dla poszczególnych modułów aplikacji;
 - static/ – Folder na pliki generowane dynamicznie przez Launcher (np. location-config.js), które wstrzykują dane aktywnej miejscowości do przeglądarki;
 - tools/ – Niezależne narzędzia deweloperskie (Edytor Właścicieli, Edytor Działek) służące do wprowadzania danych.
-

Rozdział 2. Analiza materiałów źródłowych

Podstawą merytoryczną projektu są zdigitalizowane i zinterpretowane materiały archiwalne pochodzące z dwóch niezależnych źródeł. Precyzyjne zrozumienie charakteru tych dokumentów było kluczowe dla właściwego zaprojektowania modelu danych oraz funkcjonalności całego systemu.

2.1. Dane katastralne: protokoły i mapa

Głównym źródłem danych o strukturze własnościowej są **protokoły dochodzeń miejscowych celem założenia księgi hipotecznej dla gminy katastralnej Czarna**, datowane na 1882 rok i przechowywane w zasobach **Archiwum Państwowego w Tarnowie**.

Protokoły te stanowią szczegółowy zapis stanu posiadania dla każdego właściciela w gminie. Zawierają one dane osobowe, numery posiadanych parceli (z podziałem na budowlane i gruntowe), a także niezwykle cenne opisy tekstowe dotyczące historii nabycia własności, relacji rodzinnych, transakcji oraz służebności. Każdy protokół został zeskanowany, a jego treść została wiernie przepisana do postaci cyfrowej. Te transkrypcje stały się podstawą dla danych w tabeli właściciele oraz dla autorskich sekcji analitycznych (np. powiazania_i_transakcje).

Dane przestrzenne pochodzą z oryginalnej, XIX-wiecznej mapy katastralnej gminy Czarna, która stanowiła wizualne uzupełnienie protokołów. Geometria działek, dróg, rzek i budynków została ręcznie odwzorowana w **Edytorze Działek**.

2.2. Dane genealogiczne: księgi metrykalne

Źródłem danych dla modułu genealogicznego są **księgi metrykalne (akty urodzeń, małżeństw i zgonów)** dla parafii na terenie gminy Czarna z XIX wieku. Dokumenty te przechowywane są w **Archiwum Diecezjalnym w Tarnowie**.

Księgi metrykalne pozwoliły na odtworzenie powiązań rodzinnych między mieszkańcami gminy. Zawierają one informacje o datach urodzenia i śmierci, imionach rodziców oraz danych małżonków, co umożliwiło zbudowanie wielopokoleniowych drzew genealogicznych. Dane z ksiąg zostały starannie przeanalizowane i wprowadzone do systemu za pomocą **Edytora Genealogii**. Kluczowym wyzwaniem była identyfikacja i połączenie osób z ksiąg metrykalnych z

właścicielami z protokołów katastralnych, co pozwoliło na stworzenie spójnej, zintegrowanej bazy danych społeczno–gospodarczych.

Rozdział 3. Elementy budowy systemu i wykorzystane technologie

W rozdziale opisano architekturę techniczną zaprojektowanego rozwiązania oraz uzasadnia wybór poszczególnych technologii. Przedstawiono model warstwowy systemu, składający się z frontendowej warstwy prezentacji, backendowego *API* oraz relacyjnej bazy danych, wraz z argumentacją decyzji projektowych.

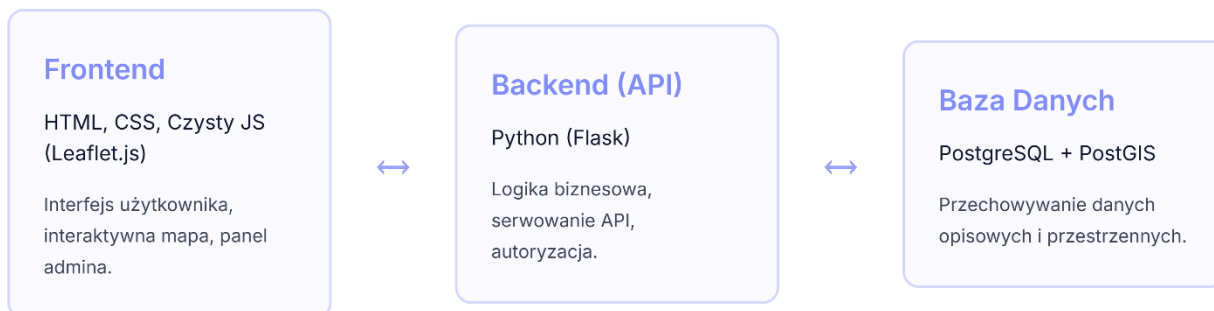
3.1. Architektura systemu

System został zaprojektowany w oparciu o sprawdzoną, trójwarstwową architekturę *full-stack*, która zapewnia wyraźny podział odpowiedzialności pomiędzy warstwą prezentacji, logiką biznesową oraz dostępem do danych. Takie rozdzielenie umożliwia niezależny rozwój poszczególnych komponentów oraz ułatwia testowanie i utrzymanie aplikacji. Szczegółowy podział odpowiedzialności poszczególnych warstw przedstawiono w tabeli 3.1. Warstwa prezentacji (*Frontend*) odpowiada za interfejs użytkownika i interakcję z mapą, warstwa logiki biznesowej (*Backend*) zarządza przetwarzaniem danych i logiką aplikacji, natomiast warstwa danych (*Database*) zapewnia trwałe przechowywanie informacji katastralnych i genealogicznych. Architektura ta zapewnia skalowalność rozwiązania oraz możliwość przyszłych rozszerzeń funkcjonalności bez konieczności przebudowy całego systemu. Schemat architektury systemu przedstawiono na rysunku 3.1.

Tabela 3.1. Warstwy architektury systemu

Warstwa	Technologia	Rola w Systemie
Frontend	HTML, CSS, JavaScript (Leaflet.js)	Odpowiada za interfejs użytkownika, wizualizację interaktywnej mapy, paneli informacyjnych oraz interakcję z użytkownikiem.
Backend (API)	Python (Flask)	Stanowi serce aplikacji. Zarządza logiką biznesową, przetwarza żądania od klienta, komunikuje się z bazą danych i udostępnia dane przez REST API.
Baza Danych	PostgreSQL + PostGIS	Przechowuje w sposób ustrukturyzowany wszystkie dane opisowe (protokoły, genealogia) oraz, dzięki rozszerzeniu PostGIS, dane przestrzenne (geometria działek, dróg, budynków).

Diagram Architektury Systemu:



Rys. 3.1.: Schemat architektury systemu

3.2. Uzasadnienie wyborów technologicznych

Dobór technologii był podyktowany chęcią stworzenia wydajnego, niezawodnego i otwartego ekosystemu:

- **backend (Python + Flask)** – wybór lekkiego frameworka Flask był podyktowany jego minimalizmem i elastycznością. Idealnie nadaje się on do tworzenia REST API, a jego prostota pozwoliła na szybkie prototypowanie i pełną kontrolę nad każdym komponentem aplikacji bez narzucania zbędnej, nadmiarowej struktury;
- **frontend (czysty JavaScript + Leaflet.js)** – rezygnacja z ciężkich frameworków (jak React czy Angular) na rzecz czystego JavaScriptu była świadomą decyzją projektową, mającą na celu skupienie się na fundamentalnych technologiach webowych i minimalizację zależności. Biblioteka **Leaflet.js** została wybrana jako lekki, a jednocześnie potężny standard do tworzenia interaktywnych map, oferujący bogaty zestaw funkcji przy zachowaniu bardzo wysokiej wydajności;
- **wizualizacja danych (D3.js)** – do realizacji zaawansowanej wizualizacji hierarchicznej, jaką jest drzewo genealogiczne, wykorzystano bibliotekę **D3.js (Data-Driven Documents)**. Jest to potężne narzędzie pozwalające na pełną kontrolę nad generowaniem dynamicznej grafiki SVG na podstawie danych. W przeciwieństwie do gotowych bibliotek do wykresów, D3.js dało swobodę implementacji autorskiego algorytmu pozycjonowania węzłów, który poprawnie modeluje złożone relacje rodzinne, co było kluczowym wymaganiem projektu;

- **baza danych (PostgreSQL + PostGIS)** – PostgreSQL to potężny, otwarty system zarządzania relacyjnymi bazami danych, znany ze swojej niezawodności i zgodności ze standardami SQL. Kluczowe dla projektu było jednak rozszerzenie **PostGIS**, które dodaje obsługę typów i funkcji geograficznych, stając się de facto standardem branżowym do przechowywania i analizy danych przestrzennych. Umożliwia ono wykonywanie złożonych zapytań geoprzestrzennych bezpośrednio w bazie danych, co jest fundamentalne dla funkcjonalności mapy.

Rozdział 4. Projekt bazy danych

Sercem systemu jest relacyjna baza danych PostgreSQL, rozszerzona o PostGIS, która pełni rolę centralnego repozytorium dla wszystkich danych projektu. Model danych został zaprojektowany z naciskiem na **integralność referencyjną**, **normalizację** (w celu unikania redundancji danych) oraz **wydajność** zapytań atrybutowych i przestrzennych. Struktura bazy została oparta na modelu encja–związek (ERD), który logicznie odwzorowuje powiązania między właścicielami, obiektami geograficznymi i danymi genealogicznymi.

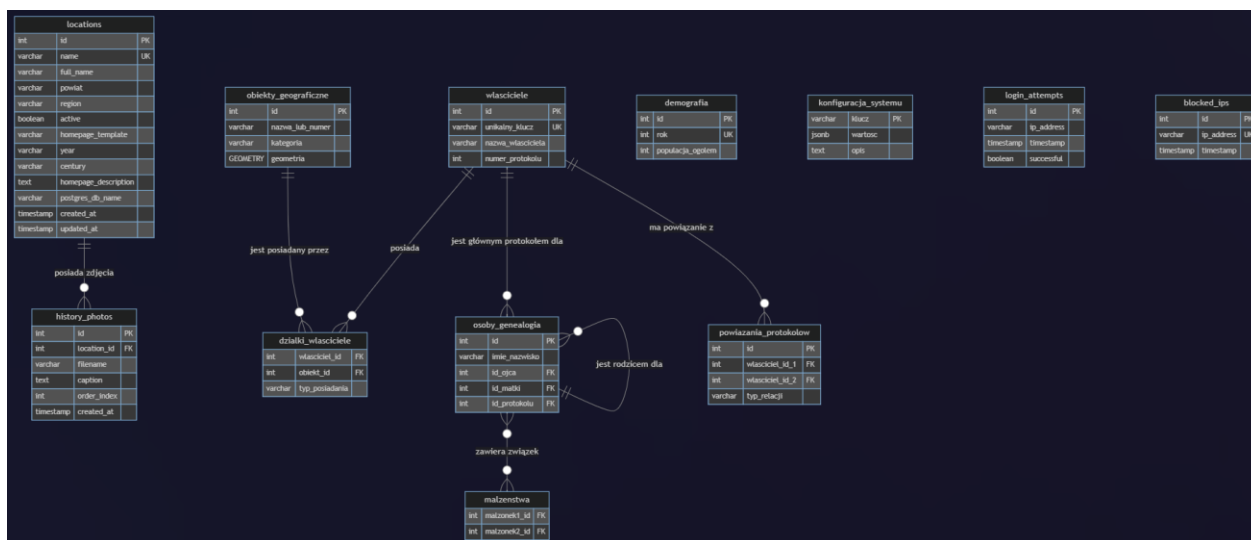
4.1. Architektura *Multitenancy* (Wielodostępność)

W celu realizacji wymogu uniwersalności narzędzia, projekt odszedł od klasycznej, monolitycznej bazy danych na rzecz architektury rozproszonej. System wykorzystuje dwie kategorie baz danych:

1. **Baza Zarządzająca (mapa_launcher_db)** – centralna baza przechowująca rejestr wszystkich miejscowości w systemie, ścieżki do plików konfiguracyjnych oraz metadane (nazwa, rok, powiat). Działa ona jako "router", wskazujący aplikacji, z którą bazą docelową ma się połączyć.
2. **Bazy Lokalizacji (mapa_{nazwa}_db)** – dla każdej dodanej miejscowości system automatycznie tworzy odizolowaną bazę danych (np. mapa_czarna_db, mapa_pilzno_db), zawierającą właściwe dane katastralne i geometryczne. Takie rozwiązanie gwarantuje bezpieczeństwo i separację danych między różnymi projektami historycznymi.

4.2. Diagram związków encji (ERD)

Na rysunku 4.1. przedstawiono diagram prezentujący kluczowe tabele w bazie danych oraz relacje (klucze obce) między nimi. Wizualizuje on, w jaki sposób dane o właścicielach, obiektach, demografii i genealogii są ze sobą połączone.



Rys. 4.1. Diagram związków encji (ERD) bazy danych

4.3. Szczegółowy opis kluczowych tabel

Schemat bazy danych na rysunku 4.1. składa się z następujących tabel:

wlasciele, **obiekty_geograficzne**, **dzialki_wlasciele**, **demografia**, **malzenstwa**, **powiazania_protokolow**, **konfiguracja_systemu**, **login_attempts**, **blocked_ips**.

Tabela wlasciele przechowuje wszystkie dane opisowe pochodzące bezpośrednio z protokołów katastralnych. Każdy rekord odpowiada jednemu protokołowi:

- **id** (*SERIAL*, *PK*) – Unikalny, auto-inkrementowany identyfikator;
- **unikalny_klucz** (*VARCHAR*, *UK*) – Czytelny dla człowieka, unikalny identyfikator (np. “Jan_Kowalski_1”), używany w URL-ach i do łączenia z danymi ze skanów;
- **nazwa_wlasciciela** (*VARCHAR*) – Imię i nazwisko właściciela;
- **numer_protokolu** (*INTEGER*) – Numer porządkowy z oryginalnego dokumentu;
- ... (pozostałe pola tekstowe) – Kolumny takie jak genealogia, historia_wlasnosci, uwagi przechowują transkrypcje z oryginalnych protokołów.

Tabela **obiekty_geograficzne** to uniwersalna tabela na wszystkie obiekty przestrzenne z mapy:

- `id` (*SERIAL, PK*) – unikalny identyfikator obiektu;
- `nazwa_lub_numer` (*VARCHAR*) – numer parceli lub nazwa obiektu (np. “Rzeka Czarna”);
- `kategoria` (*VARCHAR*) – typ obiektu, np. rolna, budowlana, droga, budynek;
- `geometria` (*GEOMETRY*) – kluczowa kolumna typu PostGIS przechowująca geometrię obiektu (punkt, linia lub poligon) w układzie współrzędnych WGS 84 (EPSG:4326).

Tabela **dzialki_wlasciciele** realizuje relację wiele–do–wielu między właścicielami a obiektami:

- `wlasciciel_id` (*INTEGER, FK*) – klucz obcy wskazujący na tabelę właściciele;
- `obiekt_id` (*INTEGER, FK*) – klucz obcy wskazujący na tabelę obiekty_geograficzne;
- `typ_posiadania` (*VARCHAR*) – rozróżnienie, czy jest to własność “rzeczywista”, czy “z protokołu”, co pozwala na analizę rozbieżności w danych.

Tabela **demografia** przechowuje historyczne dane demograficzne dla gminy:

- `rok` (*INTEGER, UK*) – kluczowy rok, dla którego wpis dotyczy;
- `populacja_ogolem, katolicy, zydzi` (*INTEGER*) – Dane liczbowe.

Tabela **osoby_genealogia** jest to centralna tabela modułu genealogicznego, przechowująca informacje o poszczególnych osobach:

- `id` (*SERIAL, PK*) – wewnętrzny identyfikator bazodanowy;
- `json_id` (*INTEGER, UK*) – oryginalne ID z pliku genealogia.json, zapewniające spójność przy imporcie;
- `id_ojca` (*INTEGER, FK*) – klucz obcy wskazujący na rekord ojca w tej samej tabeli (relacja do samego siebie);

- `id_matki` (*INTEGER, FK*) – klucz obcy wskazujący na rekord matki w tej samej tabeli;
- `id_protokolu` (*INTEGER, FK*) – opcjonalny klucz obcy wskazujący czy dana osoba jest główną postacią w którymś z protokołów w tabeli właściciele.

Tabela **malzenstwa** służy do łączenia dla relacji małżeńskich (wiele–do–wielu między osobami):

- `malzonek1_id` (*INTEGER, FK*) – klucz obcy do tabeli `osoby_genealogia`;
- `malzonek2_id` (*INTEGER, FK*) – klucz obcy do tabeli `osoby_genealogia`.

Tabela **powiazania_protokolow** realizuje relację wiele–do–wielu między protokołami. Jest kluczowa dla modułu grafu powiązań:

- `wlasciciel_id_1` (*INTEGER, FK*) – klucz obcy wskazujący na pierwszy protokół w relacji;
- `wlasciciel_id_2` (*INTEGER, FK*) – klucz obcy wskazujący na drugi protokół w relacji;
- `typ_relacji` (*VARCHAR*) – opcjonalne pole opisujące charakter relacji (np. “sprzedaż”, “dziedziczenie”).

Tabela **konfiguracja_systemu** przechowuje globalne ustawienia aplikacji w formacie klucz–wartość. Zapewnia centralne zarządzanie kluczowymi parametrami systemu:

- `klucz` (*VARCHAR, PK*) – unikalny klucz identyfikujący ustawienie (np. `map_calibration`);
- `wartosc` (*JSONB*) – wartość ustawienia przechowywana w wydajnym, binarnym formacie JSON, idealnym do przechowywania złożonych obiektów konfiguracyjnych, takich jak współrzędne mapy.;
- `opis` (*TEXT*) – opcjonalny opis przeznaczenia danego ustawienia.

Tabela **login_attempts** zapisuje każdą próbę logowania do panelu admina w celu monitorowania bezpieczeństwa i wykrywania prób ataku typu brute-force:

- ip_address (*VARCHAR*) – Adres IP, z którego nastąpiła próba;
- successful (*BOOLEAN*) – Wartość true, jeśli logowanie było udane.

Tabela **blocked_ips** przechowuje adresy IP, które zostały automatycznie zablokowane po przekroczeniu limitu nieudanych prób logowania:

- ip_address (*VARCHAR, UK*) – Zablokowany adres IP;
- reason (*TEXT*) – Powód blokady (np. automatyczna lub ręczna).

4.4. Integralność danych

Integralność referencyjna danych jest zapewniona przez użycie kluczy obcych z regułą *ON DELETE CASCADE*. Gwarantuje to, że usunięcie rekordu nadrzędnego (np. właściciela) automatycznie powoduje usunięcie wszystkich powiązanych z nim rekordów podrzędnych (np. wpisów w tabeli działki_wlasciciele), co zapobiega powstawaniu osieroconych danych i utrzymuje spójność bazy.

Rozdział 5. Implementacja ekosystemu narzędziowego

Jednym z kluczowych założeń projektu było opracowanie nie tylko aplikacji końcowej, ale kompletnego, samowystarczalnego ekosystemu zarządzania cyklem życia danych historycznych. W tym celu zaimplementowano zestaw dedykowanych narzędzi, które automatyzują, upraszczają i zabezpieczają procesy deweloperskie i administracyjne.

5.1. Launcher (`launcher/launcher_app.py`) – zintegrowane centrum dowodzenia

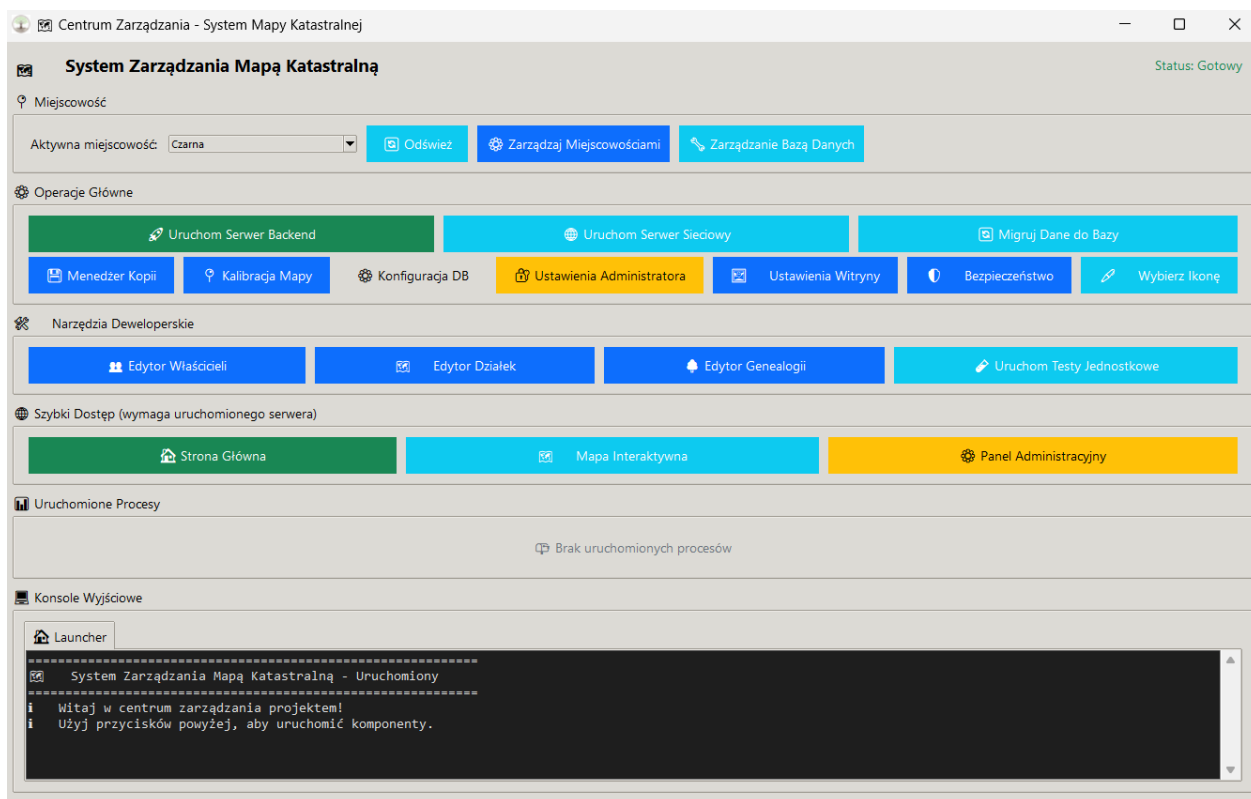
Launcher to centralna aplikacja desktopowa (*GUI*), którą przedstawiono na rysunku 5.1. Program został napisany w Pythonie z użyciem biblioteki Tkinter. Aplikacja pełni rolę zintegrowanego centrum dowodzenia całym ekosystemem, eliminując potrzebę pracy z wieloma terminalami i ręcznego wpisywania komend. Program został zaprojektowany w celu maksymalnego uproszczenia codziennej pracy z projektem.

Kluczowe funkcjonalności Launchera, to:

- **zarządzanie wieloma miejscowościami** – Launcher pełni tu rolę menedżera projektów, umożliwiając obsługę wielu niezależnych lokalizacji w ramach jednej instalacji oraz pozwala na tworzenie nowych projektów, dynamiczne przełączanie aktywnej miejscowości (co automatycznie przepina bazę danych i podmienia pliki konfiguracyjne *frontend*), a także zapewnia pełną izolację danych między różnymi gminami;
- **zarządzanie serwerem** – umożliwia uruchomienie serwera backendowego w dwóch trybach:
 - **lokalnym** – serwer dostępny tylko na lokalnej maszynie (127.0.0.1),
 - **sieciowym** – serwer dostępny dla innych urządzeń w sieci lokalnej (nasłuchuje na 0.0.0.0), z automatyczną konfiguracją reguł firewalla w systemie Windows;

- **automatyzacja migracji** – pozwala na bezpieczne przeprowadzenie jednym kliknięciem procesu migracji danych z plików JSON do bazy PostgreSQL;
- **zintegrowane narzędzia administracyjne** – menedżer kopii zapasowych – umożliwia tworzenie, przywracanie i zarządzanie kompletnymi archiwami .zip:
 - **konfiguracja bazy danych** – oferuje wbudowany edytor pliku konfiguracyjnego env,
 - **ustawienia administratora** – dedykowany panel do zarządzania bezpieczeństwem, w tym włączanie/wyłączanie autoryzacji oraz bezpieczne ustawianie hasła,
 - **konfigurator mapy** – zaawansowane okno, które łączy dwie kluczowe funkcje – **wybór pliku mapy tła** (.jpg/.png) oraz **kalibrację współrzędnych** i automatycznie kopiuje wybrany plik mapy do odpowiednich folderów (mapa/ oraz tools/parcel_editor/) oraz wyświetla jego podgląd, gwarantując spójność wizualną między edytorem a aplikacją główną,
 - **personalizacja i transparentność** – moduł pozwalający na dostosowanie identyfikacji wizualnej projektu, który umożliwia nie tylko zmianę ikony strony (favicon), ale również **dynamiczną podmianę ikony aplikacji w pasku zadań systemu Windows** (z wykorzystaniem Windows API), co pozwala na pełne dopasowanie wyglądu Launchera do aktualnie wybranej miejscowości;
- **szybki dostęp i monitoring – uruchamianie narzędzi** – zapewnia bezpośredni dostęp do wszystkich edytorów (właścicieli, działek, genealogii) oraz do uruchamiania zautomatyzowanych testów jednostkowych (pytest):
 - **bezpośrednie linki** – oferuje przyciski otwierające kluczowe widoki aplikacji (strona główna, mapa, panel administratora) bezpośrednio w przeglądarce,
 - **zarządzanie procesami** – wyświetla listę wszystkich aktywnych procesów potomnych (serwer, edytory) wraz z ich identyfikatorami (**PID**) i pozwala na ich awaryjne zatrzymanie;

- **wielozakładowa konsola** – każdy uruchomiony proces otrzymuje własną zakładkę z konsolą, w której na żywo wyświetlane są jego logi, co ułatwia diagnostykę i monitoring;
- **dynamiczna konfiguracja portów i walidacja kolizji** – w celu zapewnienia stabilności środowiska, w którym jednocześnie działa wiele procesów sieciowych, zaimplementowano system zarządzania portami TCP, w którym każda miejscowość posiada własny plik .env z definicją unikalnych portów, a launcher wyposażono w algorytm sprawdzający spójność przed uruchomieniem – system blokuje zapis ustawień w przypadku wykrycia kolizji portów, co eliminuje błędy typu *Address already in use*;
- **personalizacja identyfikacji wizualnej** – dodano moduł wyboru ikony aplikacji, który pozwala na dynamiczną podmianę grafiki w oknie oraz na pasku zadań systemu Windows (z użyciem Windows API), co pozwala na pełne dopasowanie interfejsu do konkretnej miejscowości;
- **system proaktywnego nadzoru – background guardian** – usługa monitorująca działająca w tle (jako osobny wątek), która co godzinę automatycznie weryfikuje kondycję systemu a stan prezentowany jest w oknie głównym za pomocą dynamicznego wskaźnika ("✅ System OK" lub "⚠️ Uwagi"), co pozwala to na natychmiastowe wykrycie np. błędów kodowania lub niespójności bazy bez ingerencji użytkownika.



Rys. 5.1: Główne okno aplikacji Launcher z konsolą i aktywnymi procesami

5.2. Edytor właścicieli (tools/owner_editor.py)

Edytor Właścicieli widoczny na rysunku 5.2. to samodzielna, w pełni funkcjonalna aplikacja desktopowa (GUI) napisana w Pythonie z użyciem biblioteki Tkinter. Została ona zaprojektowana jako dedykowane narzędzie dla historyków, badaczy i osób wprowadzających dane, aby w sposób ustrukturyzowany i bezpieczny zarządzać kluczowymi danymi tekstowymi projektu.

Ponieważ ręczna edycja złożonych plików JSON jest nie tylko niewygodna, ale przede wszystkim wysoce podatna na błędy (literówki, błędy składni), które mogłyby uniemożliwić późniejszą migrację danych do bazy, dlatego też problem ten rozwiązuje edytor, udostępniając przyjazny, graficzny interfejs, który prowadzi użytkownika przez cały proces i dba o poprawność formatu danych.

Kluczowe cechy i inteligentne funkcjonalności tego edytora, to:

- **nowoczesny i estetyczny interfejs** – zaprojektowany z dbałością o doświadczenie użytkownika **UX**, posiadający czytelny, responsywny układ, który automatycznie

dostosowuje się do rozdzielczości ekranu oraz używa stylizowanych widżetów (ttk), a przemyślane rozmieszczenie elementów sprawia, że praca z narzędziem jest intuicyjna i przyjemna,

- **zintegrowane centrum danych tekstowych** – edytor stanowiący centralny punkt zarządzania nie tylko protokołami, ale również danymi demograficznymi:
 - **edytor protokołów** – główny moduł aplikacji, który pozwala na edycję wszystkich pól protokołu w wygodnym formularzu (rys. 5.3), oparty na polach wielolinijkowych, takich jak “Historia własności” czy “Genealogia”, posiadających przycisk **“Powiększ”**, otwierających dedykowane, duże okno edytora tekstu, co znacząco ułatwia pracę z obszernymi opisami,
 - **zintegrowany edytor demografii** – narzędzie posiadające wbudowany, dedykowany moduł do zarządzania danymi demograficznymi, w którym po kliknięciu przycisku “Demografia”, otwiera się osobne okno przedstawione na rysunku 5.5. z tabelą, pozwalającą na **dodawanie, edycję i usuwanie wpisów demograficznych dla poszczególnych lat**, przy jednoczesnym zapisywaniu do osobnego pliku demografia.json;
- **zarządzanie skanami** – edytor przedstawiony na rysunku 5.4 jest w pełni zintegrowany z systemem plików, a sekcja “Zarządzanie Skanami” w formularzu edycji protokołu pozwala na:
 - **bezpośrednie dodawanie plików** – użytkownik może wybrać z dysku jeden lub więcej plików .jpg, które są automatycznie kopiowane do odpowiedniego folderu i numerowane,
 - **zmianę kolejności stron** – za pomocą przycisków “w górę” i “w dół” można intuicyjnie zmieniać kolejność stron protokołu, a edytor automatycznie przemianuje pliki, zachowując porządek,
 - **podgląd i usuwanie** – podwójne kliknięcie na plik na liście otwiera go w domyślnej przeglądarce obrazów;
- **automatyzacja zarządzania folderami** – narzędzie automatyzuje z folderami pracę na skanach:

- gdy tworzony jest nowy protokół, edytor **automatycznie tworzy w katalogu assets/protokoły/ nowy folder** o nazwie odpowiadającej unikalnemu kluczowi,
- gdy wpis właściciela jest usuwany, edytor wykrywa istnienie powiązanego folderu i **proponuje jego jednoczesne usunięcie**, co zapobiega pozostawianiu osieroconych danych;
- **wbudowany menedżer kopii zapasowych** – edytor posiada własnego menedżera kopii, który pozwala na tworzenie i przywracanie kompletnych archiwów .zip, z których każde z archiwów zawiera pliki .json (właściciele, demografia) oraz **cały katalog ze wszystkimi skanami protokołów i pozostałe pliki**;
- **inteligentne podpowiedzi i walidacja**:
 - **podpowiedzi kontekstowe** – w polu “Powiązania i transakcje” znajduje się ikona pomocy, która wyświetla tooltip z instrukcją formatowania linków,
 - **walidacja klucza** – system dba o poprawność i unikalność unikatowego klucza właściciela;
- **centralny eksport danych** – wszystkie dane (właściciele i demografia) są eksportowane do czystych plików .json w folderze backup/, gotowych do użycia przez skrypt migracyjny.

Kopie danych Właścicieli System Zapisów i Protokołów

Dane Wczytaj dane Kopie zapasowe ZAPISZ ZMIENI Migracja WGRUJ DANE ZAPISZ + MIGRUJ Narzędzia Demografia Zapisz Dodaj właściciela Usuń zaznaczonych Wyszukiwanie

Lp.	Imię i Nazwisko	Liczba dzieł
1	Adam Sak	18
2	Ewa z Micków Zięta i Michał Zięta	9
3	Anna Micek	10
4	Barbara Sakowa	11
5	Józef Tryba (spadkobiercy Majjenny z Pychów)	17
6	Michał Pijet	2
7	Magdalena Murasłowa z Seków	19
8	Jan Pych	20
9	Wojciech Para (syn Michała)	28
10	Józef Para	20
11	Adam Tabudziński	3
12	Michał Sak	37
13	Majjenna z Jarmogów 1'Sakowa 2'Barrasłowa 3'Parzywa (i Józef Para)	11
14	Jan Para (syn Michała)	11
15	Wojciech Jaramski	19
16	Jan Para (syn Józefa)	20
17	Wawrzyniec Knapik	6
18	Adam Knapik	3
19	Piotr Augustyn	18
20	Adam Basiarz (spadkobierca)	16
21	Piotr Augustyn (spadek z dóbr tabudzińskich)	2
22	Jedroń Pędrek	3
23	Jan Sak z Borow	5
24	Karol Augustyn	18
25	Wojciech Płaza	12
26	Alfred Łupa	2
27	Stanisław Łupa (spadkobiercy)	23
28	Stanisław Łupa (spadkobiercy, syn Klemensa)	29
29	Wawrzyniec Barnaś	14
30	Rozalia Falciniewiczowa i Błażej Falciniewicz	3
31	Majjenna z Knapków Kusłowa	10
32	Ewa z Urbanów Parzywa	12
33	Wojciech Cygan	23
34	Wincenty Nicot	24
35	Jan Łabak (reprezentowany przez opiekuna Wojciecha Cygana)	17
36	Majjenna Perzłowa (wstępuje w imieniu spadkobierców)	12
37	Salomon Grzesiński	2
38	Jan Jędrzej	28
39	Jędrzej Micek	13

Rys 5.2. Widok sekcji edycji właścicieli w bazie danych

Kopia Tęczy: Adam Sak			
Kontyngent			
Unikalny Identyfikator: Adam Sak			
Dane Właściciela			
Lp. 1			
Imię i Nazwisko: Adam Sak			
Data protokołu: Dzień		Miesiąc (słownie)	Rok
10		Junego	1882
Numer domu: 17			
Miejsce protokołu: Włazno			
Dziaki (numery oddzielone przecinkami)			
Dziaki budowlane (z protokołu)			Powiększ
1/1			
Dziaki rolne (z protokołu)			Powiększ
801/2, 801/3, 801/4, 801/5, 801/6, 801/7, 801/8, 804, 805, 806, 807/1, 807/2, 808, 809, 818, 819, 820			
Dziaki budowlane (zeczywiste)			Powiększ
1/1			
Dziaki rolne (zeczywiste)			Powiększ
801/2, 801/3, 801/4, 801/5, 801/6, 801/7, 801/8, 804, 805, 806, 807/1, 807/2, 808, 809, 818, 819, 820			
Dodatkowe Informacje			
Genealogia:			Powiększ
Ojciec: Mikołaj Sak (zmarły w 1874 roku)			
Historia posiadania działek:			Powiększ
Z posiadłością tą nie jest połączona żadna służebność, a podprawy w rozszerzeniu swoim majątkiem nie jest ograniczony. Parcele grunt. Lk. 801/2, 801/3, 801/4, 801/5, 801/6, 801/7, 801/8 nabyłem kontraktem kupna, sprawę z daty Machowa 14 grudnia 1878r. od Henryki hr. Kuczkowskiej z korpusu tabularnego dóbr Czarna i takowe w posiadanie objąłem, reszta powyższych parcel była własnością mojego ojca Mikołaja Saka, w roku 1874 zmarłym, po którym atoli pertraktacja spadku dotąd przeprowadzona nie została. Upraszam aby z posiadłości powyższej utworzone zostały dwa ciała hipoteczne w osobnych wykazach. 1. Do ciała hipotecznego pierwszego należeć mają parcele gruntowe 801/2, 801/3, 801/4, 801/5, 801/6, 801/7, 801/8, a prawo własności takowego ma być zainstalowane dla mnie podpisanego Adama Saka. 2. Do ciała hipotecznego drugiego należeć będzie renta powyższej posiadłości, a prawo własności takowego zainstalowane zostanie dla nie objętej masy spadkowej Saka Mikołaja.			
Ciąg niniejszy:			Powiększ
Ciąg dalszy spisany dnia 23 lutego 1882 Staje Kartator masy spadkowej Henryki hr. Kuczkowskiej p. Franciszek Jakubowski i oświadczając: Z uwagi iż parcele gruntowe 801/2, 801/3, 801/4, 801/5, 801/6, 801/7, 801/8 przez Adama Saka, więc powstały przez kontraktem kupna-przedzają z kompletu tabularnego dóbr Czarna zostały nabyte, z korpusu tabularnego tychże dóbr dotyczących nie są wyłączone, przeto nie zgadzam się, aby z takowych ciał hipoteczne utworzone zostały i by prawo własności tychże parcel dla Adama Saka zostało zainstalowane, lecz wnoszę, aby parcele powyższe jako korpus tab. dóbr Czarny zainstalowane, do karty stanu posiadania tego korpusu tabularnego zostały zapisane.			
Wypokreślenie/Służebność:			Powiększ
Posiadanie i transakcje ? Nabył część gruntów (parcele 801/2 do 801/8) od Henryki hr. Kuczkowskiej w 1878 roku.			

Rysunek 5.3. Widok sekcji edycji danego właściciela

Historia Tworcy Adam Sak	
Dodatkowe informacje	
Genealogia:	Powiększ
Ojciec: Mikołaj Sak (zmarły w 1874 roku)	
Historia posiadania działek:	Powiększ
Z posiadłości ta nie jest połączona żadna służebność, a podprawy w rozszerzeniu swoim majątkiem nie jest ograniczony. Parcele grunt. Lx. 801/2, 801/3, 801/4, 801/5, 801/6, 801/7, 801/8 nabyłem kontrastem kupna, sprawą z daty Machowa 14 grudnia 1878r. od Henryki hr. Kuczkowskiej z korpusu tabularnego dóbr Czarna i takowe w posiadanie objąłem, resztą powyższych parcel była własnością mojego ojca Mikołaja Saka, w roku 1874 zmarłym, po którym atoli pertrakcja spadku dotąd przeprowadzona nie została. Upraszam aby z posiadłości powyżej utworzone zostały dwa ciała hipoteczne w osobnych wykazach: 1. Do ciała hipotecznego pierwszego należeć mają parcele gruntowne 801/2, 801/3, 801/4, 801/5, 801/6, 801/7, 801/8, a prawo własności takiego ma być zarejestrowane dla mnie podpisanego Adama Saka. 2. Do ciała hipotecznego drugiego należeć będzie renta powyższej posiadłości, a prawo własności takiego zarejestrowane zostanie dla nie objętej mojej spadkowej Saka Mikołaja.	
Ciąg dalszy tematu:	Powiększ
Któryś dzisiejszy syn dnia 23 lutego 1882 Stąpi kurator masy spadkowej Henryki hr. Kuczkowskiej p. Handziak Jakubowski i oświadcza: Z uwagi że parcele gruntowne 801/2, 801/3, 801/4, 801/5, 801/6, 801/7, 801/8 przez Adama Saka, więc powstały przez kontraktom kupna-sprzedaży z kompletem tabularnego dóbr Czarna zostały nabyte, a z korpusu tabularnego tychże dóbr dotyczących nie są wyłączone, przeto nie zgadzam się, aby z takowych dla hipoteczne utworzone zostały i by prawo własności tychże parcel dla Adama Saka zostało zarejestrowane, lecz życzę, aby parcele powyższe jako korpusu tab. dóbr Czarny zarejestrowane, do karty stanu posiadania tego korpusu tabularnego zostały zapisane.	
Wyposażenie/Służebność:	Powiększ
Powiązania i transacje :	
Nabył część gruntów (parcelę 801/2 do 801/8) od Henryki hr. Kuczkowskiej w 1878 roku.	
Analiza	Powiększ
Interpretacja i wnioski	
Protokół formalnie rozdziela majątek Adama Saka na dwie części: grunty nabyte osobiście oraz nieuregulowany spadek po ojcu, Mikołaju. Kluczowym elementem jest spor z dobrami dworskimi (tabularnymi), reprezentowanymi przez kuratora, który blokuje wpisanie do księgi wieczystej działek nabytych od hr. Kuczkowskiej. Ukazuje to problemy prawne związane z obrotem ziemią pomiędzy właścicielami a dworem.	
Zatwierdzanie Skanum i Protokołu	
1.jpg 2.jpg 3.jpg 4.jpg	
Dodaj stan... Usuw błędności	

Rys. 5.4. Sekcja zarządzania skanami z możliwością zmiany kolejności stron

Rok	Populacja	Katolicy	Żydzi	Inni	Opis
1882	555	505	50	0	Dane szacunkowe na podstawie pierwszych spisów po uruchomieniu k.
1885	442	0	0	0	Dane sprzed budowy kościoła.

Rys. 5.5 Dedykowany moduł do edycji danych demograficznych

5.3. Edytor działek (tools/parcel_editor/parcel_editor_app.py)

Edytor działek widoczny na rysunku 5.6. to zaawansowane narzędzie zaprojektowane specjalnie dla potrzeb procesu digitalizacji obiektów przestrzennych z historycznej mapy katastralnej. Składa się z lekkiego serwera backendowego w Pythonie (Flask) oraz interaktywnego interfejsu frontendowego w przeglądarce, opartego na bibliotece **Leaflet.js** i jej rozszerzeniu **Leaflet.PM (Geoman)**.

Umożliwia on rozwiązanie problemu ręcznego wprowadzania współrzędnych geograficznych dla setek poligonów, linii i punktów, które jest nie tylko niepraktyczne i czasochłonne, ale również obarczone ogromnym ryzykiem błędu. Edytor działek zastępuje ten proces w pełni wizualnym, intuicyjnym interfejsem *“point-and-click”*, który pozwala na precyzyjne i szybkie odwzorowanie geometrii obiektów z oryginalnej mapy.

Do kluczowych cech i funkcjonalności edytora należą:

- **wizualny interfejs “Co widzisz, to dostajesz” (WYSIWYG)** – narzędzie, które po uruchomieniu serwuje stronę internetową, na której historyczna mapa katastralna jest wyświetlana jako interaktywny podkład, a użytkownik może swobodnie powiększać i przesuwać mapę, aby dokładnie zidentyfikować granice działek, przebieg dróg czy lokalizację budynków;

- **zaawansowane narzędzia rysowania:**

- **rysowanie poligonów** – dedykowane przyciski pozwalają na rysowanie obiektów powierzchniowych, takich jak działki rolne, budowlane, lasy czy pastwiska, po wybraniu który użytkownik klika na mapie kolejne wierzchołki, a narzędzie na bieżąco rysuje kształt,

- **rysowanie linii i punktów** – edytor obsługuje również rysowanie obiektów liniowych (drogi, rzeki) oraz punktowych (budynki, kapliczki), automatycznie dostosowując typ tworzonej geometrii do wybranej kategorii,

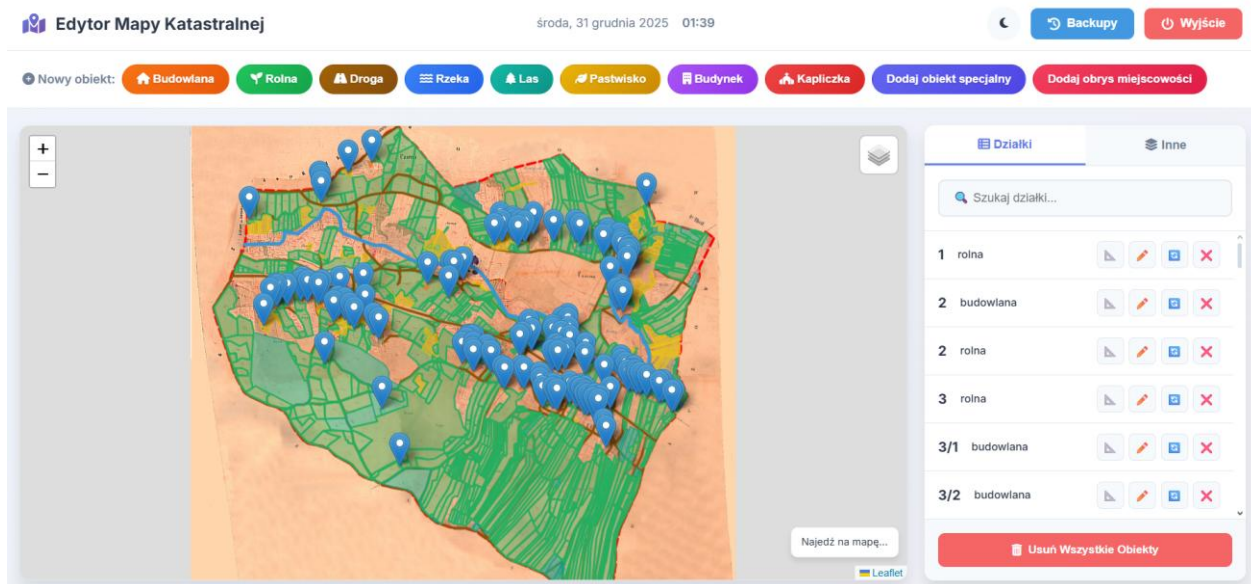
- **definiowanie granic obrębu (kontur)** – narzędzie zostało wyposażone w specjalny tryb "Rysuj Kontur Miejscowości", a użytkownik za jego pomocą obrysowuje granice administracyjne wsi, zaś system zapisuje tę geometrię jako unikalny obiekt, który służy później do wizualnego wyróżnienia obszaru na mapie głównej oraz do automatycznego obliczenia całkowitej powierzchni gminy (w hektarach i km²),

- **inteligentne narzędzia pomocnicze** – podczas rysowania poligonów dostępne są funkcje takie jak “Cofnij Punkt” i “Zakończ”, co daje pełną kontrolę nad procesem tworzenia geometrii,

- **rysowanie konturu miejscowości** – specjalny tryb pozwalający na obrysowanie granic administracyjnych wsi, a utworzony poligon służy jako baza do obliczeń powierzchniowych w panelu statystyk;

- **kompleksowe zarządzanie obiektami:**
 - **panel boczny** – wszystkie narysowane obiekty są na bieżąco wyświetlane na liście w panelu bocznym, pogrupowane w logiczne zakładki (Działki, Inne Obiekty),
 - **interaktywna lista** – umożliwia najechanie kursorem na obiekt na liście podświetla go na mapie, a kliknięcie centruje na nim widok,
 - **edycja atrybutów** – użytkownik może w każdej chwili zmienić numer/nazwę obiektu bezpośrednio z poziomu listy,
 - **edycja geometrii** – dedykowany przycisk “Edytuj geometrię” pozwala na modyfikację kształtu już istniejącego obiektu przez przeciąganie jego wierzchołków;
- **wbudowany menedżer kopii zapasowych** – podobnie jak edytor właścicieli, to narzędzie posiada własny system zarządzania backupami, pozwala on na:
 - **tworzenie kopii zapasowych** – jednym kliknięciem można stworzyć archiwum `parcels_data_backup_RRRRMMDD_GGMMSS.json`, zawierające aktualny stan wszystkich narysowanych obiektów,
 - **przywracanie stanu** – umożliwia wczytanie danych z dowolnego, wcześniejszego pliku kopii zapasowej, co chroni przed przypadkową utratą danych,
 - **zarządzanie plikami** – pozwala na przeglądanie i usuwanie starych kopii zapasowych;
- **eksport w standardzie GeoJSON** – cała sesja rysowania jest na bieżąco zapisywana, a finalny wynik pracy jest przechowywany w pliku `backup/parcels_data.json`. Dane są zapisywane w formacie **GeoJSON**, który jest otwartym standardem w świecie

systemów informacji geograficznej (*GIS*). Zapewnia to pełną interoperacyjność i gotowość danych do użycia przez skrypt migracyjny, który przetwarza je i importuje do bazy PostGIS.



Rys. 5.6.: Interfejs główny Edytora działek z widocznym menedżerem kopii zapasowych

5.4. Edytor genealogii (tools/genealogy_editor/editor_app.py)

Edytor genealogii przedstawiony na rysunku 5.7. to wyspecjalizowane narzędzie webowe, zaprojektowane do zarządzania złożonymi powiązaniami rodzinnymi w ramach społeczności gminy Czarna. Podobnie jak Edytor Działek, opiera się on na architekturze z lekkim backendem w Pythonie (Flask) i dynamicznym interfejsem w przeglądarce.

Rozwiązuje on problem ręcznego modelowania i utrzymywania spójności wielopokoleniowych relacji rodzinnych w pliku tekstowym lub arkuszu kalkulacyjnym, które jest niezwykle trudne i podatne na błędy (np. niespójne ID, pętle w relacjach). Edytor genealogii dostarcza ustrukturyzowany, tabelaryczny interfejs, który upraszcza tworzenie i walidację powiązań między osobami.

Kluczowe cechy i funkcjonalności:

- **interfejs tabelaryczny z inteligentnym grupowaniem** – wszystkie osoby w bazie genealogicznej są przedstawione w czytelnej, interaktywnej tabeli, dla której w celu

ułatwienia nawigacji, zaimplementowano mechanizm automatycznego grupowania osób według rodów (nazwisk), a każda rodzina jest prezentowana w zwijanej sekcji, co pozwala na szybkie przeglądanie i analizę poszczególnych linii genealogicznych;

- **zarządzanie osobami (CRUD)** – edytor zapewnia pełne możliwości zarządzania danymi osób:

- **dodawanie** – formularz “Dodaj Nową Osobę” pozwala na wprowadzenie wszystkich kluczowych danych, takich jak imię, nazwisko, płeć, lata życia czy powiązany protokół,

- **edycja** – każdy wpis w tabeli można edytować za pomocą dedykowanego formularza modalnego,

- **usuwanie** – bezpieczne usuwanie osób z bazy z prośbą o potwierdzenie;

- **intuicyjne tworzenie relacji** – największą siłą edytora jest uproszczenie procesu tworzenia powiązań rodzinnych:

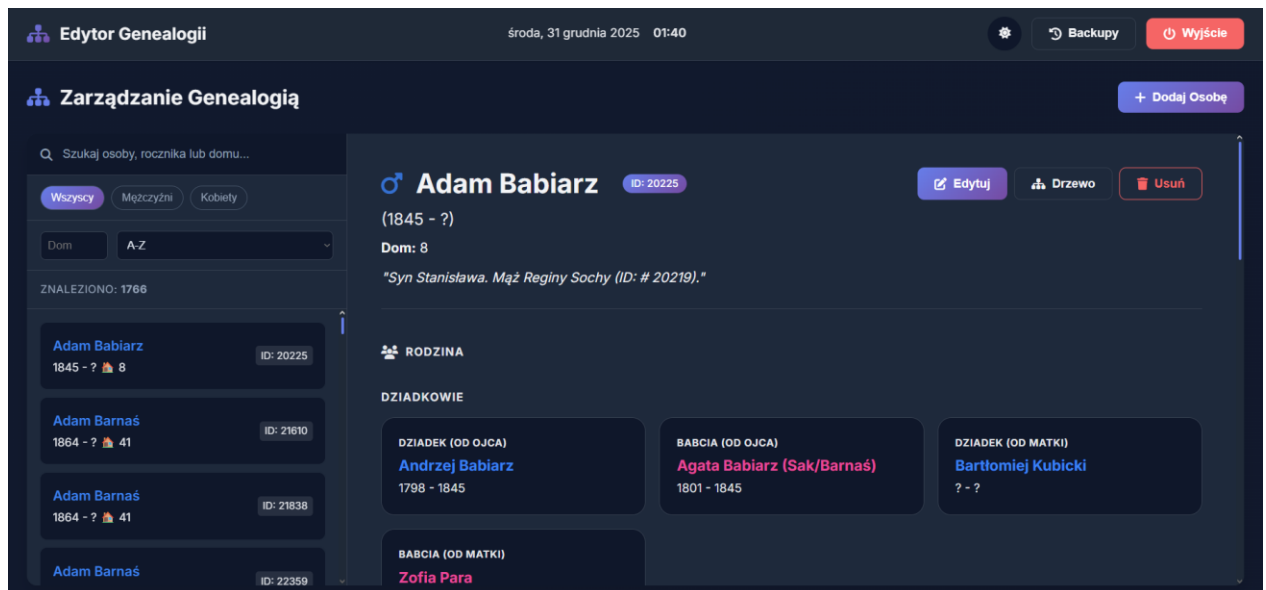
- **autouzupełnianie** – pola formularza służące do definiowania relacji rodzinnych (“Ojciec”, “Matka”, “Małżonek”) zostały zintegrowane z dynamiczną listą podpowiedzi (<datalist>), a podczas wpisywania imienia lub nazwiska, edytor na bieżąco sugeruje pasujące osoby z bazy, co eliminuje konieczność pamiętania i ręcznego wpisywania skomplikowanych identyfikatorów,

- **walidacja i symetryzacja** – backend edytora dba o spójność danych. Przy zapisie weryfikuje, czy podane *ID* rodziców i małżonków istnieją w bazie, a dodatkowo, automatycznie symetryzuje relacje małżeńskie – przypisanie męża do żony i automatycznie tworzy odwrotne powiązanie żony z mężem;

- **wbudowany menedżer kopii zapasowych** – narzędzie posiada własny system zarządzania kopiami zapasowymi, który pozwala na tworzenie, przywracanie i usuwanie

archiwalnych wersji pliku genealogia.json oraz zapewnia bezpieczeństwo danych i możliwość powrotu do poprzednich stanów w razie pomyłki;

- **zintegrowana wizualizacja drzewa** – edytor jest bezpośrednio połączony z modulem wizualizacji:
 - dla każdej rodziny (grupy w tabeli) dostępny jest przycisk **“Drzewo”**, który dynamicznie generuje i wyświetla w oknie modalnym interaktywne drzewo genealogiczne dla danego rodu, wykorzystując zaawansowany algorytm renderowania oparty na bibliotece D3.js,
- **eksport do pliku JSON** – całość danych genealogicznych jest zapisywana w ustrukturyzowanym pliku backup/genealogia.json, który następnie służy jako źródło dla głównego skryptu migracyjnego.



Rys. 5.7. Interfejs główny Edytora Genealogii z grupowaniem rodów i formularzem edycji

Rozdział 6. Implementacja backendu i procesu migracji

Backend stanowi kluczowy element systemu, odpowiadający za zarządzanie danymi, obsługę *API* oraz logikę biznesową. W tym rozdziale przedstawiono szczegóły implementacji warstwy serwerowej oraz proces migracji danych z narzędzi deweloperskich do finalnej bazy danych.

6.1. Serwer *API* (backend/app.py)

Backend aplikacji pełni rolę mózgu operacji. Został zaimplementowany w Pythonie przy użyciu mikroframeworka Flask. Jego główne zadania to:

- **udostępnianie REST *API*** – serwuje dane z bazy PostgreSQL przez zdefiniowane endpointy (np. `/api/wlasciciele`, `/api/dzialki`), które są zwracane w formacie JSON, gotowym do użycia przez aplikację frontendową;
- **serwowanie aplikacji frontendowej** – Flask serwuje również wszystkie pliki statyczne (HTML, CSS, JS, obrazy), dzięki czemu cały projekt działa jako jedna, spójna aplikacja bez potrzeby konfiguracji zewnętrznych serwerów jak Apache czy Nginx, a przy serwowaniu strony mapy, backend dynamicznie wstrzykuje do kodu HTML aktualne dane konfiguracyjne (współrzędne, zoom) pobrane z bazy danych;
- **logika biznesowa** – zawiera logikę do obsługi bardziej złożonych zapytań, np. generowanie danych do grafu powiązań czy statystyk;
- **zarządzanie konfiguracją** – przy starcie serwer wczytuje do pamięci globalne ustawienia systemowe z tabeli `konfiguracja_systemu`, co zapewnia, że wszystkie komponenty aplikacji korzystają z tych samych, aktualnych parametrów.

Kluczowe endpointy *API*

Do najważniejszych publicznych endpointów *API*, które dostarczają dane do aplikacji klienckiej należą:

- **GET /api/wlasciele**
 - **opis** – zwraca listę wszystkich właścicieli wraz z podstawowymi informacjami o ich działkach,
 - **użycie** – główna lista właścicieli na mapie interaktywnej;
- **GET /api/dzialki**
 - **opis** – zwraca wszystkie obiekty geograficzne z geometrią w standardowym formacie GeoJSON, a ponieważ zapytanie jest realizowane bezpośrednio w bazie danych, to zapewnia wysoką wydajność,
 - **użycie** – renderowanie wszystkich działek, dróg i budynków na mapie;
- **GET /api/wlasciciel/<unikalny_klucz>**
 - **opis** – zwraca szczegółowe dane dla jednego, konkretnego właściciela na podstawie jego unikalnego klucza,
 - **użycie** – widok szczegółowego protokołu;

- **GET /api/stats**
 - **opis** – zwraca zagregowane dane statystyczne, w tym ogólne podsumowania, rankingi największych właścicieli oraz dane demograficzne,
 - **użycie** – zasilanie w dane całej sekcji statystyk;
- **GET /api/graph-data**
 - **opis** – generuje dane potrzebne do wizualizacji grafu powiązań, w którym węzły to właściciele, a krawędzie są tworzone na podstawie linków zdefiniowanych w danych,
 - **użycie** – renderowanie grafu powiązań;
- **GET /api/genealogia/<unikalny_klucz>**
 - **opis** – na podstawie klucza protokołu, algorytm przeszukuje bazę danych, aby znaleźć całą połączoną sieć rodzinną (a nie tylko bezpośrednich przodków/potomków) i zwraca ją w formacie gotowym do wizualizacji,
 - **użycie** – generowanie dynamicznych drzew genealogicznych;

Panel administracyjny korzysta z osobnego zestawu endpointów pod ścieżką /api/admin/..., które implementują pełne operacje *CRUD* (*Create, Read, Update, Delete*) dla wszystkich kluczowych tabel.

6.2. Mechanizmy bezpieczeństwa

W celu zabezpieczenia panelu administracyjnego oraz ochrony integralności danych, zaimplementowano następujące mechanizmy bezpieczeństwa:

konfigurowalna autoryzacja – dostęp do panelu administratora może być włączony lub wyłączony za pomocą zmiennej `ADMIN_AUTH_ENABLED` w pliku `.env`, co pozwala to na elastyczne dostosowanie poziomu bezpieczeństwa w zależności od środowiska (deweloperskie vs. produkcyjne);

Haszowanie haseł – hasła administratorów nie są przechowywane w formie czystego tekstu, lecz zamiast tego, używana jest biblioteka `werkzeug.security` do generowania i weryfikacji bezpiecznych haszy haseł (z użyciem soli), co uniemożliwia ich odczytanie nawet w przypadku wycieku danych z pliku konfiguracyjnego;

Ochrona przed atakami Brute-Force – system monitoruje próby logowania, a po przekroczeniu progu pięciu nieudanych prób logowania z tego samego adresu *IP* w ciągu 15 minut, adres ten jest automatycznie blokowany;

Zarządzanie zablokowanymi *IP* – zaimplementowano mechanizm przechowywania zablokowanych adresów *IP* w bazie danych, a launcher oferuje dedykowany interfejs “Menedżer Bezpieczeństwa”, który pozwala administratorowi na:

- przeglądanie logów z próbami logowania;
- ręczne odblokowywanie adresów *IP*, w tym awaryjne odblokowanie localhost w przypadku przypadkowej samoblokady.

6.3. Cykl życia danych i proces migracji

Dane w projekcie przechodzą przez precyzyjnie zdefiniowany, kontrolowany cykl życia, co zapewnia ich spójność i integralność.

1. **Konfiguracja mapy** – administrator, za pomocą **Kalibratora Mapy** w Launcherze, wprowadza współrzędne mapy. Dane te są zapisywane jednocześnie do pliku backup/map_config.json oraz do tabeli konfiguracja_systemu w bazie danych.

2. **Dane źródłowe (backup/*.json)** – wszystkie dane wprowadzane są za pomocą dedykowanych edytorów, które zapisują je w postaci ustrukturyzowanych plików JSON. Edytor Działek korzysta z map_config.json, aby zapewnić spójność rysowanych obiektów z finalną mapą. Pliki te stanowią “źródło prawdy” i podstawę do tworzenia kopii zapasowych.

3. **Migracja (backend/migrate_data.py)** – skrypt migracyjny wczytuje pliki JSON, czyści docelowe tabele w bazie danych, a następnie importuje dane, transformując geometrię do formatu PostGIS (*WKT*). Proces ten jest idempotentny, co oznacza, że jego wielokrotne uruchomienie zawsze prowadzi do tego samego, spójnego stanu bazy.

4. **Udostępnianie (API)** – działający serwer Flask wczytuje konfigurację mapy z bazy, a następnie udostępnia dane o obiektach i właścicielach przez endpointy API.

5. **Zarządzanie (panel admina)** – panel administracyjny pozwala na modyfikację danych bezpośrednio w bazie.

6. **Backup (eksport)** – funkcja eksportu w panelu admina odwraca proces migracji, generując pliki JSON z aktualnego stanu bazy danych, co zamyka cykl życia danych.

6.4. Obsługa błędów i kody statusu API

API zostało zaprojektowane z myślą o przewidywalnej obsłudze błędów. Klient front-endowy może polegać na standardowych kodach statusu http w celu odpowiedniej reakcji na wynik żądania. Szczegółowe zestawienie kodów statusu wraz z ich znaczeniem przedstawiono w tabeli 6.1.

Tabela 6.1. Kody statusu HTTP API i ich znaczenie

Kod statusu	Znaczenie	Przykład użycia
200 OK	Sukces. Żądanie zostało pomyślnie przetworzone.	GET /api/wlasciciele
201 Created	Sukces. Nowy zasób został pomyślnie utworzony.	POST /api/admin/wlasciciele
400 Bad Request	Błąd klienta. Żądanie zawiera nieprawidłowe lub brakujące dane.	POST /api/admin/login bez hasła
401 Unauthorized	Błąd uwierzytelnienia. Nieprawidłowy login lub hasło.	POST /api/admin/login z błędnym hasłem
403 Forbidden	Brak uprawnień. Użytkownik jest zidentyfikowany, ale nie ma dostępu.	Próba logowania z zablokowanego adresu IP
404 Not Found	Nie znaleziono. Żądany zasób nie istnieje.	GET /api/wlasciciel/nieistniejacyklucz
409 Conflict	Konflikt. Próba utworzenia zasobu, który już istnieje.	POST /api/admin/wlasciciele z istniejącym kluczem
500 Internal Server Error	Błąd serwera. Wystąpił nieoczekiwany błąd po stronie backendu.	Błąd połączenia z bazą danych

6.5. Skrypt migracyjny (backend/migrate_data.py)

Skrypt migracyjny jest to kluczowy skrypt jednorazowego użytku, który zasila system w dane. Jego proces działania jest następujący:

1. **Odczyt danych źródłowych** – wczytuje dane z plików `owner_data_to_import.json` i `parcels_data.json`.
2. **Czyszczenie bazy danych** – usuwa wszystkie istniejące dane z tabel, aby zapewnić spójność i uniknąć duplikatów przy ponownej migracji.
3. **Import danych** – wstawia rekordy do odpowiednich tabel (właściciele, obiekty_geograficzne, itd.), dbając o poprawne typy danych i relacje.
4. **Transformacja geometrii** – Konwertuje współrzędne geograficzne z formatu JSON do formatu WKT (*Well-Known Text*) wymaganego przez PostGIS, co umożliwia wykonywanie zapytań przestrzennych.

Rozdział 7. Implementacja aplikacji frontendowej

Interfejs użytkownika został zbudowany z użyciem fundamentalnych technologii webowych – HTML5, CSS3 oraz czystego JavaScriptu (ES6+), co zapewniło wysoką wydajność i minimalną liczbę zależności.

7.1. Mapa interaktywna (mapa/)

Głównym komponentem aplikacji jest interaktywna mapa przedstawiona na rysunku 7.1., stanowiąca centralny punkt eksploracji danych katastralnych. Została ona zaprojektowana jako zaawansowany, wielofunkcyjny interfejs, który łączy w sobie wizualizację danych przestrzennych z bogatymi narzędziami analitycznymi i nawigacyjnymi.

7.1.1. Podstawowa architektura mapy

Sercem modułu jest biblioteka **Leaflet.js**, która zarządza renderowaniem wszystkich warstw graficznych. Dane o obiektach geograficznych i właścicielach są dynamicznie pobierane z backendowego *API*, co zapewnia, że prezentowane informacje są zawsze aktualne. Mapa została skonfigurowana z precyzyjnymi ograniczeniami, uniemożliwiającymi użytkownikowi wyjście poza zdefiniowany obszar gminy (*maxBounds*) oraz nadmierne przybliżanie lub oddalanie widoku (*minZoom*, *maxZoom*), co gwarantuje płynność działania i kontekst geograficzny.

7.1.2. Interfejs użytkownika i kluczowe funkcjonalności

Interfejs użytkownika został podzielony na trzy główne strefy – centralną mapę oraz dwa zwijane panele boczne, co pozwala na maksymalizację przestrzeni roboczej w zależności od potrzeb.

Panel lewy – **właściciele** – jest dedykowany do przeglądania i analizy danych o właścicielach protokołów:

- **lista właścicieli** – domyślnie prezentuje listę wszystkich właścicieli, posortowaną według numeru protokołu,
- **interaktywność** – kliknięcie na nazwisko właściciela natychmiast przenosi użytkownika do szczegółowego widoku jego protokołu,

- **najeżdżanie kursorem** na wpis na liście dynamicznie podświetla na mapie wszystkie działki należące do danego właściciela, co pozwala na szybką identyfikację jego posiadłości,
- **filtrowanie i sortowanie** – daje użytkownikowi do dyspozycji zaawansowane opcje sortowania listy – alfabetycznie (A–Z), według numeru protokołu (domyślnie) oraz według liczby posiadanych działek (od największej),
- **wyszukiwarka kontekstowa** – pozwala na błyskawiczne filtrowanie listy zarówno po nazwisku, jak i po numerze protokołu,
- **rozdzielenie własności** – umożliwia wizualizację dwóch typów własności – rzeczywistej (potwierdzonej) oraz według protokołu (która może zawierać historyczne błędy), a dedykowane przyciski pozwalają na podświetlenie na mapie obu tych zbiorów działek, zaś specjalny przycisk przełączający umożliwia dynamiczną zmianę widoku.

Panel prawy – **działki i obiekty** – służący do eksploracji obiektów geograficznych został podzielony na logiczne zakładki:

- **działki** – nie tylko grupuje działki rolne, budowlane, lasy i pastwiska, ale posiada także dodatkowe filtry pozwalające na włączanie i wyłączanie widoczności poszczególnych kategorii,
- **infrastruktura** – prezentuje obiekty liniowe, takie jak drogi i rzeki,
- **specjalne** – gromadzi obiekty punktowe o szczególnym znaczeniu, takie jak budynki mieszkalne, kapliczki oraz inne ważne obiekty (np. dworzec kolejowy, młyn),
- **interaktywność** – podobnie jak w panelu lewym, najeżdżanie na obiekt na liście podświetla go na mapie, a kliknięcie centruje na nim widok.

Centralny widok mapy tworzą:

- **legenda interaktywna** – znajdująca się w lewym dolnym rogu i zwijana nie tylko objaśnia kolorystykę poszczególnych kategorii obiektów, ale również pozwala na dynamiczne włączanie i wyłączanie ich widoczności na mapie,

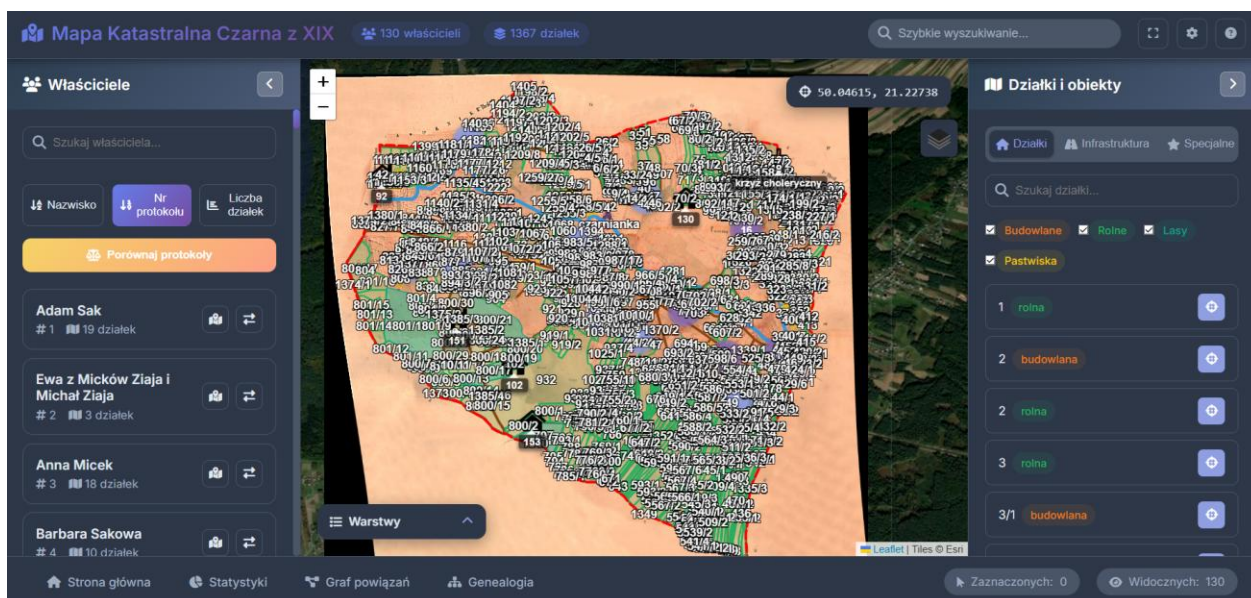
- **przełącznik warstw** – zaimplementowany w prawym górnym rogu warstw bazowych umożliwia płynne przełączanie się między podkładem z dziewiętnastowieczną mapą katastralną a współczesnymi widokami: **mapą drogową (OpenStreetMap)** oraz **ortofotomapą (satelitarną)**, co pozwala na bezpośrednie porównanie historycznego układu gruntów z obecnym stanem terenu,
- **obsługa współwłasności** – inteligentnie obsługuje przypadki, gdy jedna działka ma wielu właścicieli – kliknięcie na taki obiekt na mapie powoduje wyświetlenie okna dialogowego z listą wszystkich współwłaścicieli, umożliwiając użytkownikowi wybór protokołu, który chce przeanalizować,
- **informacje kontekstowe** – w prawym górnym rogu na bieżąco wyświetlane są precyzyjne współrzędne geograficzne kursora, zaś w prawym dolnym rogu znajduje się ukryta legenda, która pojawia się dynamicznie podczas podświetlania działek, aby objaśnić kolorystykę (np. przy porównywaniu stanu rzeczywistego i z protokołu).

Na dolny pasek narzędziowy składają się:

- **uniwersalna wyszukiwarka** – centralnie umieszczona wyszukiwarka pozwala na przeszukiwanie całej bazy danych – zarówno właścicieli, jak i obiektów – i prezentuje wyniki w formie dynamicznej listy,
- **liczniki stanu** – informuje o liczbie **“Zaznaczonych”** obiektów (gdy jakaś grupa działek jest podświetlona) oraz **“Widocznych”** właścicieli (co jest szczególnie przydatne podczas filtrowania).
- **narzędzia aplikacji:**
 - **tryb pełnoekranowy** – umożliwia przełączenie aplikacji w tryb pełnoekranowy,
 - **ustawienia** – otwiera okno modalne, w którym użytkownik może zresetować widok do stanu początkowego lub przełączyć motyw kolorystyczny na ciemny, który jest zapamiętywany i automatycznie stosowany we wszystkich

modułach aplikacji (mapa, protokół, statystyki), zapewniając spójne doświadczenie wizualne,

- **pomoc** – wyświetla okno z informacjami o skrótach klawiszowych i wskazówkami dotyczącymi użytkowania,



Rys. 7.1. Główny widok interaktywnej mapy z panelami bocznymi i przełącznikiem warstw

7.2. Panel administracyjny (admin/)

Panel administracyjny, przedstawiony na rysunku 7.2, to zaawansowana, w pełni funkcjonalna aplikacja webowa, zaprojektowana w architekturze *Single Page Application (SPA)*. A to oznacza, że cała logika interfejsu (przełączanie między widokami, edycja danych, wyświetlanie formularzy) odbywa się dynamicznie w jednym pliku HTML (admin.html), bez potrzeby przeładowywania strony. Takie podejście, które zastąpiło pierwotną koncepcję wielu oddzielnych stron (*login*, *dashboard*, edytory), zapewnia płynność działania i nowoczesne doświadczenie użytkownika.

Panel stanowi centralny punkt zarządzania danymi bezpośrednio w bazie PostgreSQL, oferując administratorowi intuicyjny i bezpieczny interfejs do wykonywania wszystkich kluczowych operacji.

Do kluczowych cech i funkcjonalności panelu administracyjnego należą:

- **zunifikowany interfejs (SPA)** – wszystkie moduły zarządzania – od pulpitu, przez edytory właścicieli, obiektów, demografii, aż po genealogię – są dostępne w jednym, spójnym interfejsie bez konieczności nawigacji między różnymi stronami;
- **kompleksowy interfejs *CRUD*** – zapewnia pełen zestaw operacji (Tworzenie, Odczyt, Aktualizacja, Usuwanie) dla wszystkich kluczowych danych:
 - **zarządzanie właścicielami** – intuicyjny formularz widoczny na rysunku 7.3. pozwala na dodawanie i edycję protokołów, a jego kluczową funkcją jest możliwość przypisywania działek z interaktywnej listy rozwijanej, co eliminuje konieczność ręcznego wpisywania numerów z pamięci i minimalizuje ryzyko pomyłek;
 - **zarządzanie demografią** – przejrzysty interfejs tabelaryczny umożliwia łatwe dodawanie i edytowanie historycznych danych demograficznych dla poszczególnych lat;
- **zaawansowany moduł genealogiczny:**
 - **inteligentne grupowanie** – tabela z listą osób automatycznie grupowanych według rodów, co znacząco ułatwia nawigację po złożonych strukturach rodzinnych,
 - **wizualizacja drzewa** – zaawansowany algorytm do renderowania interaktywnych drzew genealogicznych, który potrafi poprawnie wizualizować złożone relacje, np. pokazując córkę zarówno w drzewie genealogicznym jej rodziców, jak i w drzewie rodziny jej męża, co odzwierciedla realia historycznych powiązań;
- **narzędzia administracyjne:**
 - **dynamiczny backup danych** – bardzo ważna funkcja wykorzystywana przy migracji infrastruktury, tworzeniu kopii zapasowych lub gdy potrzebny jest lokalny zrzut danych bez konieczności ręcznego przepisywania – administrator

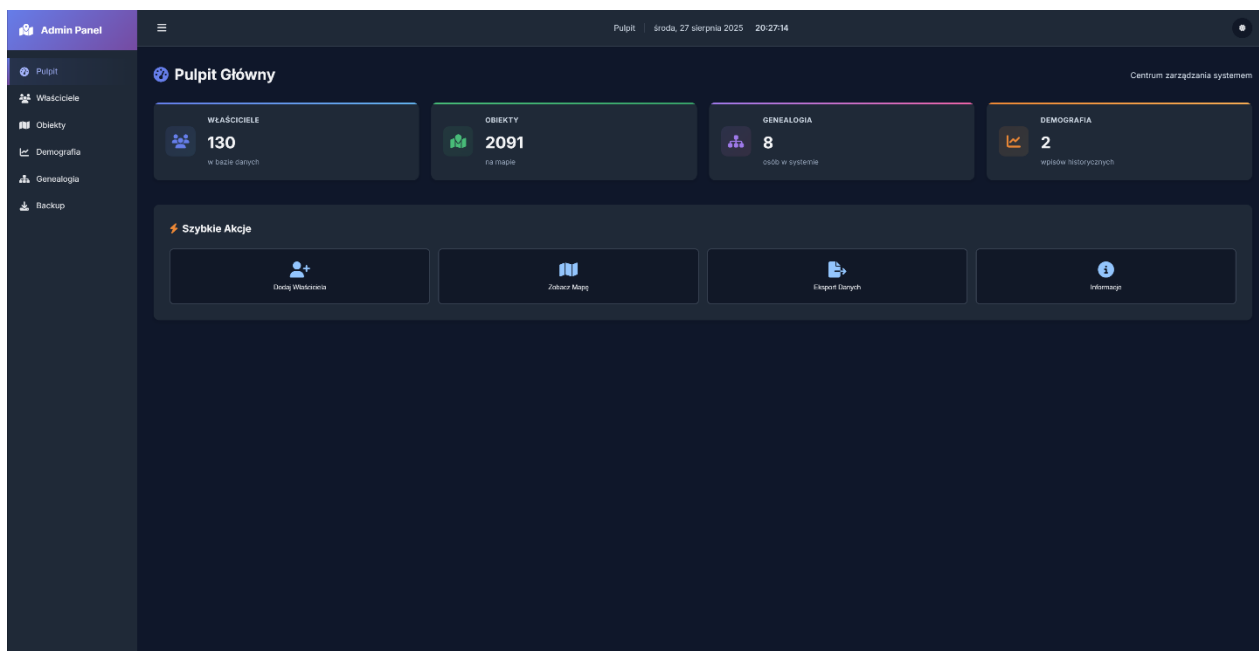
może w każdej chwili kliknąć przycisk “**Ściągnij Backup**”, który uruchamia proces eksportu aktualnego stanu całej bazy danych do archiwum .zip.,

- **bezpieczeństwo** – dostęp do tego panelu jest chroniony przez konfigurowalny system logowania, opisany szczegółowo w rozdziale dotyczącym bezpieczeństwa;

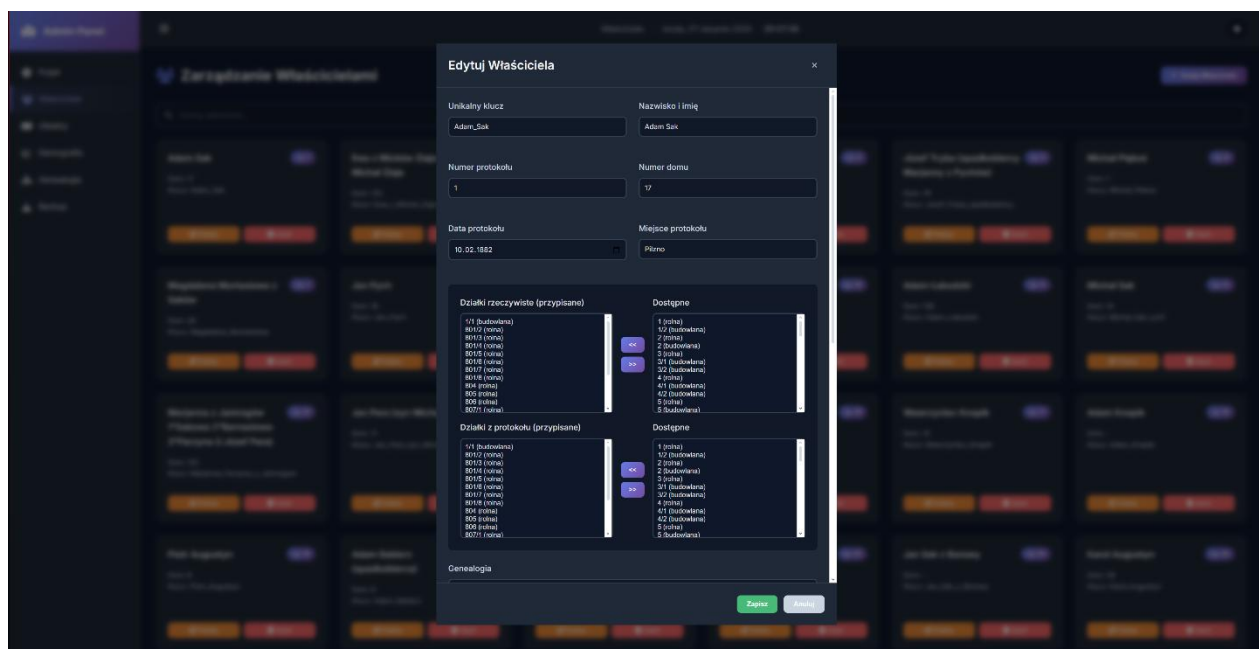
- **dbałość o doświadczenie użytkownika (UI/UX):**

- **estetyczny interfejs** – panel został zaprojektowany z dbałością o detale wizualne, aby praca z danymi była nie tylko efektywna, ale i przyjemna,

- **dynamiczne elementy** – interfejs zawiera takie “smaczki”, jak dynamicznie aktualizowana data i zegar w czasie rzeczywistym, co dodaje mu profesjonalizmu.



Rys. 7.2. Pulpit główny (dashboard) Panelu Administracyjnego



Rys. 7.3. Formularz edycji protokołu w Panelu Administracyjnym z interaktywnym polem wyboru działek

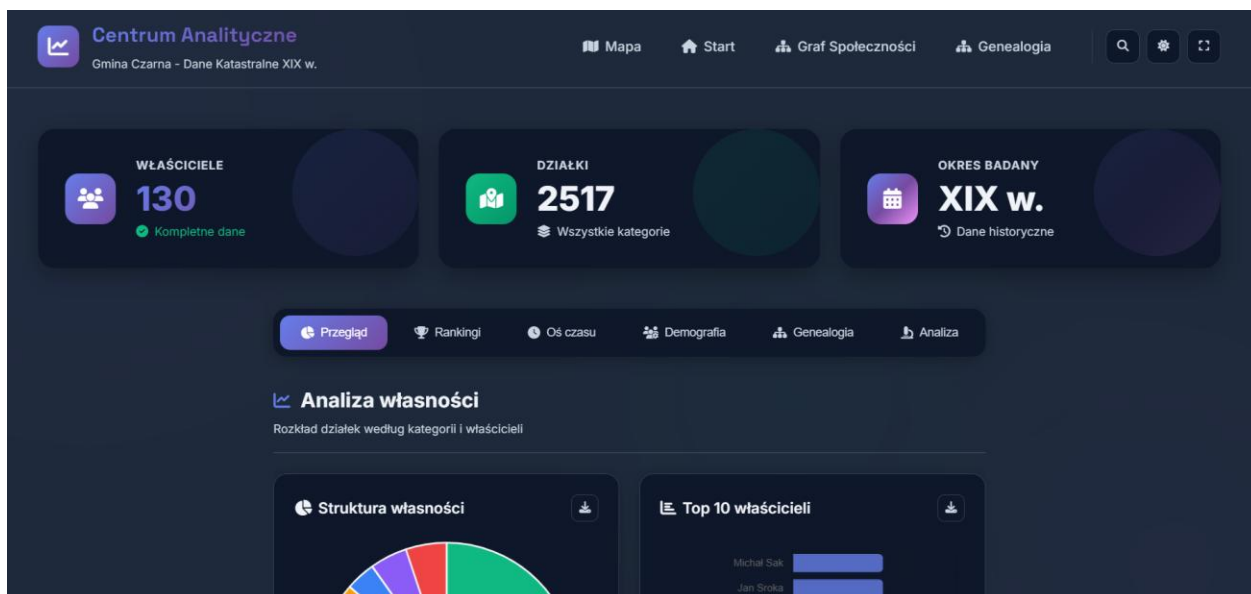
7.3. Centrum analityczne – statystyki i rankingi (wlascciele/stats.html)

Strona statystyk przedstawiona na rysunku 7.4. została zaprojektowana jako nowoczesne **Centrum Analityczne (*Dashboard*)**, które prezentuje zagregowane dane w przystępnej i interaktywnej formie. Wykorzystuje bibliotekę **Chart.js** do dynamicznego generowania wykresów, co pozwala na wizualną reprezentację złożonych danych.

Kluczowe komponenty centrum analitycznego, to:

- **pulpit główny (*Hero Section*)** – na górze strony znajdują się kluczowe wskaźniki efektywności KPI (*Key Performance Indicators*), takie jak wartości całkowitej liczby zdigitalizowanych właścicieli i działek, które są animowane, co przyciąga uwagę użytkownika i podkreśla skalę zgromadzonych danych;
- **Interaktywne wykresy:**
 - **Struktura własności** – wykres kołowy (*doughnut chart*) prezentuje procentowy udział poszczególnych kategorii działek (rolne, budowlane, lasy, etc.) w całej gminie,
 - **ranking właścicieli** – wykres słupkowy (*bar chart*) wizualizuje 10 największych posiadaczy gruntów, pozwalając na szybką identyfikację najbardziej wpływowych postaci w regionie,
 - **wykres demograficzny** – wykres liniowy (*line chart*) przedstawia zmiany w populacji gminy na przestrzeni lat, z podziałem na grupy wyznaniowe (katolicy, żydzi), co pozwala na analizę trendów demograficznych;
- **kalendarz aktywności spisowej** – unikalna wizualizacja w formie “mapy ciepła” (heatmap), prezentującej intensywność spisywania protokołów w poszczególnych dniach, na której kolor komórki odpowiada liczbie protokołów utworzonych danego dnia, co pozwala na analizę rytmu pracy komisji katastralnej;
- **interaktywne wykresy i analizy:**
 - **struktura własności i Infrastruktura** – wykresy prezentujące udział poszczególnych kategorii działek oraz nowa funkcja automatycznego obliczania długości dróg i rzek w kilometrach;

- 48



Rys. 7.4.: Widok Centrum Analitycznego z interaktywnymi wykresami i rankingami

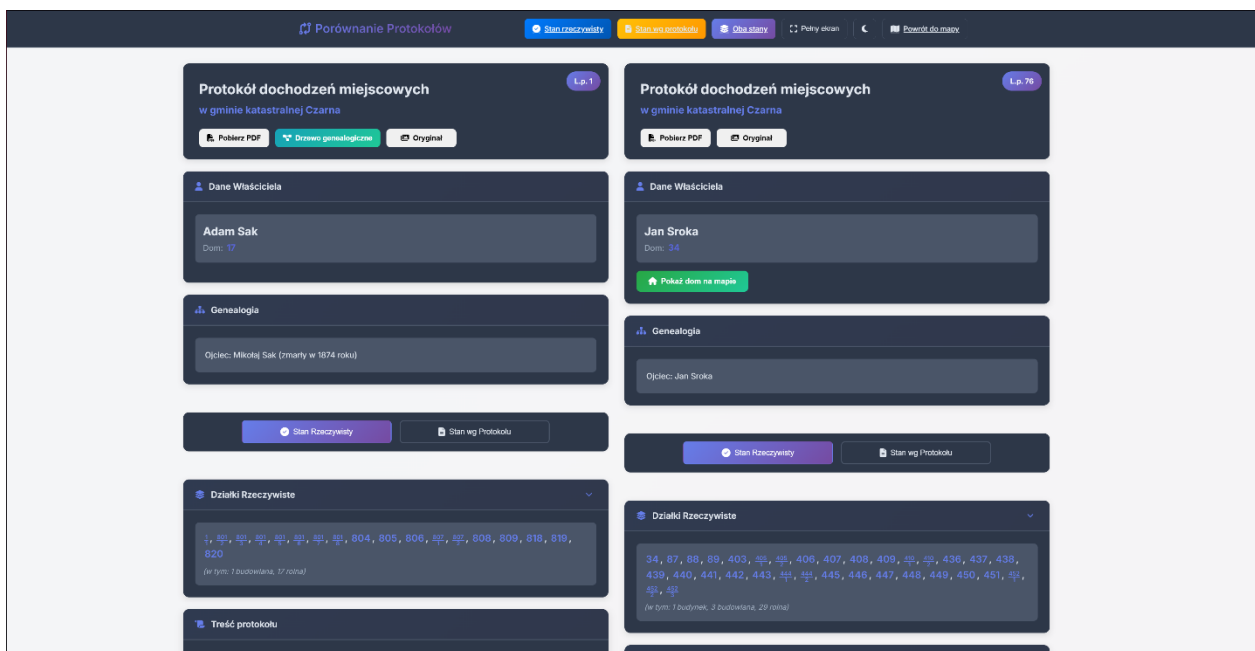
Uniwersalna wyszukiwarka – strona jest wyposażona w globalną wyszukiwarkę, która w czasie rzeczywistym filtruje wszystkie widoczne komponenty (rankingi, oś czasu) i podświetla znalezione frazy.

7.3.1. Porównywarka protokołów (własciciele/compare.html)

Narzędzie to zostało opracowane w celu ułatwienia szczegółowej analizy porównawczej. Użytkownik, będąc na mapie, może wejść w tryb porównywania, wybrać dwóch dowolnych właścicieli, a następnie jest przenoszony do dedykowanego widoku (rys. 7.5).

Porównywarkę protokołów tworzą:

- **układ dwukolumnowy** – strona prezentuje dane z dwóch protokołów w przejrzystym układzie “obok siebie”, co ułatwia dostrzeganie różnic i podobieństw w stanie posiadania, danych genealogicznych czy historii transakcji;
- **wspólna nawigacja do mapy** – przyciski na górnym pasku pozwalają na jednoczesną wizualizację na mapie działek obu porównywanych właścicieli, z rozróżnieniem na stan rzeczywisty i z protokołu;
- **eksport do PDF** – każdy z protokołów może być indywidualnie wyeksportowany do pliku PDF, co jest przydatne do archiwizacji lub dalszej analizy offline.



Rys. 7.5. Interfejs Porównywarki Protokołów w układzie dwukolumnowym

7.3.2. Widok szczegółowy protokołu (właściciele/protokol.html)

Widok szczegółowy protokołu przedstawiony na rysunkach 7.6., 7.7., 7.8. jest centralnym punktem prezentacji danych opisowych dla pojedynczego właściciela. Strona ta została zaprojektowana w celu jak najwierniejszego odwzorowania informacji zawartych w oryginalnych, dziewiętnastowiecznych dokumentach, a jednocześnie wzbogacenia ich o kontekst cyfrowy i analityczny. Źródłem danych dla tego widoku są protokoły dochodzeń miejscowych celem założenia księgi hipotecznej, pochodzące z Archiwum Państwowego w Tarnowie.

Z punktu widzenia struktury i prezentacji danych strona została podzielona na logiczne, tematyczne karty, które ułatwiają nawigację i przyswajanie informacji:

- **nagłówek protokołu** – na samej górze strony wyświetlane są kluczowe metadane z oryginalnego dokumentu, takie jak data i miejsce sporządzenia protokołu (np. 10.02.1882, Pilzno), co natychmiast osadza dokument w historycznym kontekście;
- **dane właściciela** – sekcja prezentująca podstawowe informacje o osobie, której dotyczy protokół, w tym imię, nazwisko oraz numer domu;

- **sekcje analityczne (wkład własny)** – aby zwiększyć wartość analityczną surowych danych, wprowadzone zostały dodatkowe, autorskie sekcje interpretacyjne, które nie występowały w oryginalnych dokumentach:

genealogia – zwięzłe podsumowanie najbliższych relacji rodzinnych (małżonek, dzieci, rodzice), które stanowi wprowadzenie do pełnego drzewa genealogicznego, a zawarte tu dane są efektem integracji informacji z protokołów katastralnych oraz ksiąg metrykalnych z Archiwum Diecezjalnego;

– **współ własność / służebność** – interpretacja zapisów dotyczących praw osób trzecich do majątku;

– **powiązania i transakcje** – wylistowanie innych osób wspomnianych w protokole, co jest kluczowe dla budowania grafu powiązań;

– **interpretacja i wnioski** – autorska analiza i podsumowanie treści protokołu, które wyjaśnia jego znaczenie w kontekście historycznym i prawnym;

- **stan posiadania (rzeczywisty vs. wg protokołu)** – interfejs wyraźnie rozdziela dwa stany posiadania, prezentując listy numerów działek dla każdego z nich, co pozwala na łatwe zidentyfikowanie rozbieżności między stanem faktycznym a zapisami w katastrze;

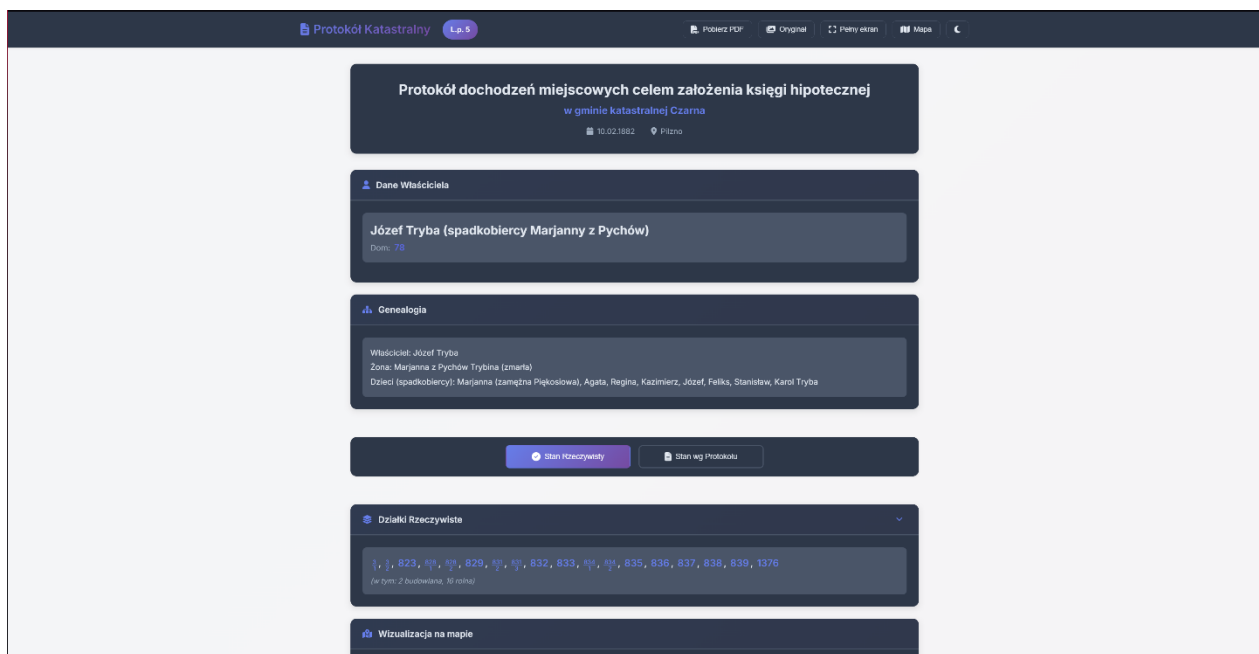
- **treść protokołu** – w tej sekcji znajduje się wierna transkrypcja oryginalnej treści dokumentu, z zachowaniem historycznego języka i stylu.

Jeśli chodzi o interaktywność i realizujące ją narzędzia, to widok protokołu jest w pełni interaktywny i zintegrowany z resztą ekosystemu:

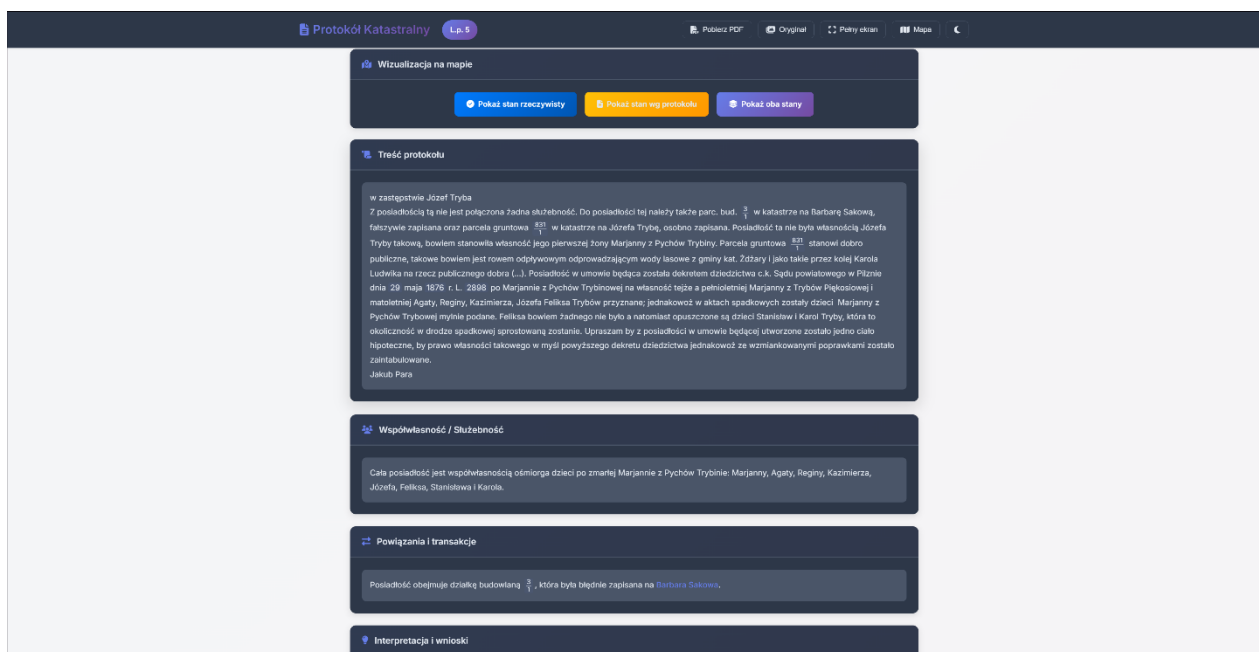
- **wizualizacja na mapie** – użytkownik ma do dyspozycji przyciski, które pozwalają na natychmiastowe podświetlenie na interaktywnej mapie wszystkich działek powiązanych z danym protokołem (zarówno w stanie rzeczywistym, jak i wg protokołu) oraz lokalizacji domu właściciela;

- **drzewo genealogiczne** – przycisk **“Pokaż drzewo genealogiczne”** dynamicznie generuje i wyświetla w oknie modalnym pełną, interaktywną wizualizację sieci rodzinnej powiązanej z daną osobą;

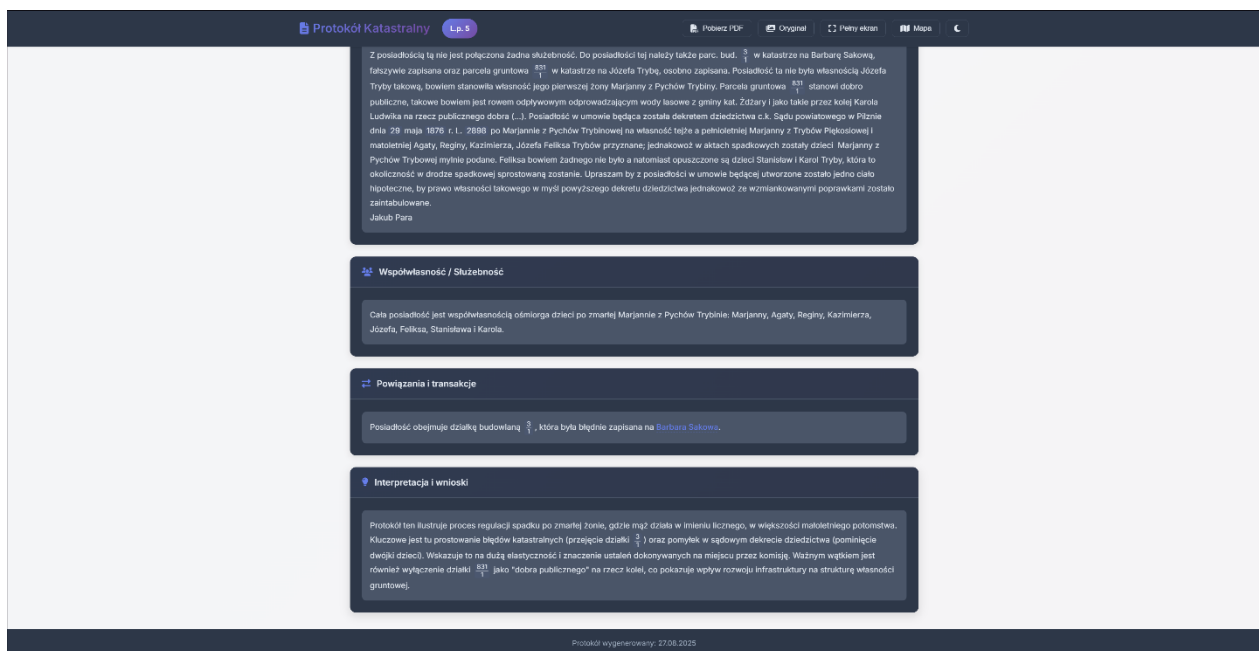
- **narzędzia dodatkowe** – górny pasek nawigacyjny oferuje narzędzia takie jak przełączenie widoku w tryb pełnoekranowy dla wygodniejszej analizy, eksport całego protokołu do pliku PDF oraz bezpośredni dostęp do zdigitalizowanych skanów oryginalnego dokumentu.



Rys 7.6. Widok szczegółowy protokołu – sekcja nagłówkowa i analityczna



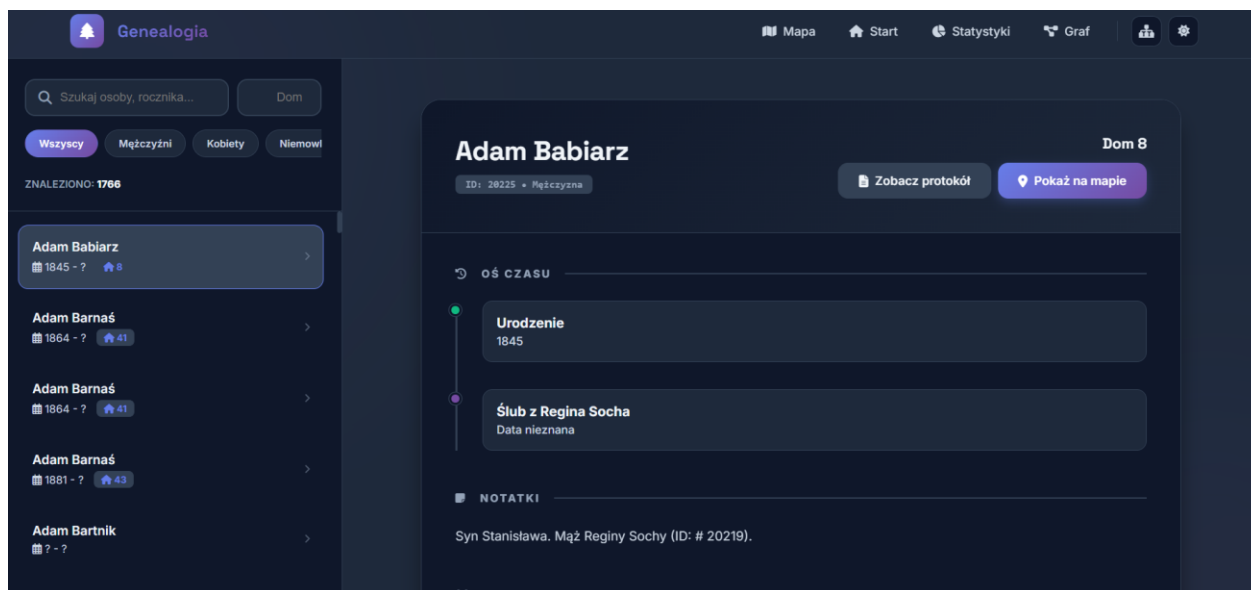
Rys. 7.7. Widok szczegółowy protokołu – sekcja stanu posiadania i transkrypcji



Rys. 7.8. Widok szczegółowy protokołu – sekcja interpretacji

7.4. Moduł genealogiczny (genealogia/ i graf/) – wizualizacja sieci społecznych

Moduł genealogiczny, zaprezentowany na rysunku 7.9, stanowi jeden z najbardziej innowacyjnych komponentów projektu, przekształcając płaskie dane z protokołów w dynamiczny, personocentryczny system analizy powiązań rodzinnych mieszkańców Gminy Czarna w XIX wieku. Kluczowym osiągnięciem modułu jest zastosowanie układu indeksu osób, który pozwala na błyskawiczną nawigację po bazie danych i automatyczne generowanie kontekstowej mapy relacji dla wybranej osoby. Daje to precyzyjny, a zarazem czytelny wgląd w mikrostrukturę społeczną całej społeczności bez ryzyka chaosu wizualnego.



Rys. 7.9. Interfejs modułu genealogicznego z widokiem indeksu osób (lewa kolumna) oraz kartą profilu z wizualizacją najbliższej rodziny (prawa kolumna).

7.4.1. Indeks Osób i profil rodziny (genealogia.html)

Zamiast statycznego grafu, wprowadzono interaktywny **indeks osób**, który stanowi centralne źródło wiedzy o mieszkańcach. Kluczowe funkcjonalności tego widoku to:

- **układ dwukolumnowy** – lewy panel zawiera listę osób z wyszukiwarką i filtrami (płeć, wiek, numer domu), natomiast prawy wyświetla pełny profil wybranego mieszkańca;
- generowanie dokumentacji personalnej (karta rodziny pdf) – zaimplementowano funkcję eksportu danych osoby do formatu PDF a skrypt backendowy agreguje metadane, oś czasu życia oraz graf relacji najbliższej rodziny, tworząc gotowy do druku raport;
- inter-modułowa kontrola stanu serwera – wdrożono mechanizm sprawdzający dostępność API przed przejściem z widoku osoby do widoku protokołu, dzięki czemu, jeśli serwer backendowy nie jest uruchomiony, użytkownik otrzymuje czytelny komunikat zamiast błędu połączenia, co znacząco poprawiło stabilność nawigacji;
- **wizualizacja najbliższej rodziny** – autorski algorytm generuje dla każdej osoby dynamiczny schemat relacji, który w przeciwieństwie do klasycznych drzew, skupia się na otoczeniu takich osób najbliższych jak:
 - **dziadkowie i rodzice** – reprezentujący linię wstępną,

- **rodzeństwo i kuzynostwo** – wizualizacja relacji bocznych (rzadko spotykana w standardowych systemach),
- **małżonkowie i dzieci** – reprezentujący linię zstępną;
- **oś czasu i notatki** – każda karta zawiera chronologiczny wykaz wydarzeń (urodzenie, śluby, zgon) oraz transkrypcje uwag z oryginalnych ksiąg metrykalnych.

7.4.2. Mapa Rodzin (mapa_rodzin.html)

Jest to moduł dedykowany analizie makroskopowej, służący do badania całych grup społecznych:

- **automatyczne grupowanie w linie rodowe** – system analizuje bazę i łączy osoby o tym samym nazwisku w odrębne „gałęzie” na podstawie wspólnych przodków;
- **identyfikacja protoplastów** – moduł wskazuje najstarszą znaną osobę w danej linii, pozwalając śledzić rozwój rodziny na przestrzeni pokoleń;
- **interaktywne miniatury drzew rodowych** – po wybraniu danej linii rodowej, system generuje dedykowane drzewo genealogiczne dla całego rodu.

7.4.3. Graf powiązań społeczno-ekonomicznych (graf/graf.html)

Widok grafu powiązań (rys. 7.10) stanowi makroskopowe narzędzie analityczne, którego celem jest wizualizacja sieci społeczno–ekonomicznych w Gminie Czarna. W odróżnieniu od Drzewa Genealogicznego, które skupia się na relacjach rodzinnych, graf ten modeluje powiązania między protokołami katastralnymi, odzwierciedlając transakcje, dziedziczenie i inne interakcje udokumentowane w historycznych źródłach. Został on zaimplementowany przy użyciu biblioteki **Vis.js Network**, która specjalizuje się w renderowaniu złożonych, dynamicznych grafów z wykorzystaniem symulacji fizycznych.

Do kluczowych aspektów implementacji grafu powiązań należą

- **model danych grafu:**
 - **węzły (*Nodes*)** – każdy węzeł na grafie reprezentuje jeden unikalny protokół katastralny, a jego etykieta zawiera nazwisko właściciela oraz numer porządkowy, jego wielkość jest dynamicznie skalowana i wprost proporcjonalna do liczby jego połączeń (stopnia wierzchołka), co pozwala na natychmiastową identyfikację **węzłów centralnych (hubów)** – kluczowych postaci w społeczności, które brały udział w największej liczbie interakcji;
 - **krawędzie (*Edges*)** – krawędzie łączące węzły są generowane automatycznie na podstawie linków zdefiniowanych w polu powiazania_i_transakcje w bazie danych i odzwierciedlają one formalne oraz nieformalne relacje, które mogły wynikać z transakcji kupna–sprzedaży, dziedziczenia gruntów, powiązań rodzinnych czy innych interakcji zapisanych w protokołach;
- **interaktywność i analityka:**
 - **panel kontrolny** – użytkownik ma do dyspozycji zaawansowany, zwijany panel boczny, który stanowi centrum sterowania wizualizacją i umożliwia:
 - **wyszukiwanie i selekcję** – błyskawiczne odnajdywanie i zaznaczanie konkretnych węzłów na grafie;
 - **filtrowanie dynamiczne** – możliwość filtrowania grafu w czasie rzeczywistym, np. przez ukrywanie węzłów o małej liczbie połączeń, co pozwala na skupienie analizy na najbardziej wpływowych postaciach;
 - **zmianę układu** – przełączanie między różnymi algorytmami pozycjonowania węzłów, w tym układem hierarchicznym;
 - **symulacja fizyczna** – domyślnie graf jest renderowany z włączoną symulacją fizyczną, a jego węzły odpychają się od siebie, zaś krawędzie działają jak sprężyny, co sprawia, że cała struktura organicznie układa się w przestrzeni, co sprawia, że taki układ w sposób naturalny ujawnia klastry, czyli grupy gęsto powiązanych ze sobą protokołów, które mogą reprezentować, np. blisko

współpracujące rodziny lub sąsiadów – użytkownik może w każdej chwili włączyć lub wyłączyć tę symulację, aby “zamrozić” układ do dalszej analizy;

- **eksploracja powiązań** – interfejs został zaprojektowany z myślą o intuicyjnej eksploracji:

- **podświetlanie przy najechaniu** – najechanie kursorem na dowolny węzeł powoduje jego podświetlenie wraz ze wszystkimi bezpośrednimi sąsiadami, co ułatwia śledzenie bezpośrednich relacji;
- **szczegółowe informacje (*Tooltip*)** – po najechaniu na węzeł wyświetlana jest etykieta z dodatkowymi informacjami;
- **nawigacja do protokołu** – **podwójne kliknięcie** na dowolny węzeł natychmiast przenosi użytkownika do szczegółowego widoku protokołu danej osoby, co pozwala na płynne przejście od analizy makro (cała sieć) do analizy mikro (pojedynczy dokument).



Rys. 7.10. Wizualizacja sieci społeczno–ekonomicznych w module grafu powiązań

7.4.4. Drzewo genealogiczne (genealogia.html oraz logika w protokol.js)

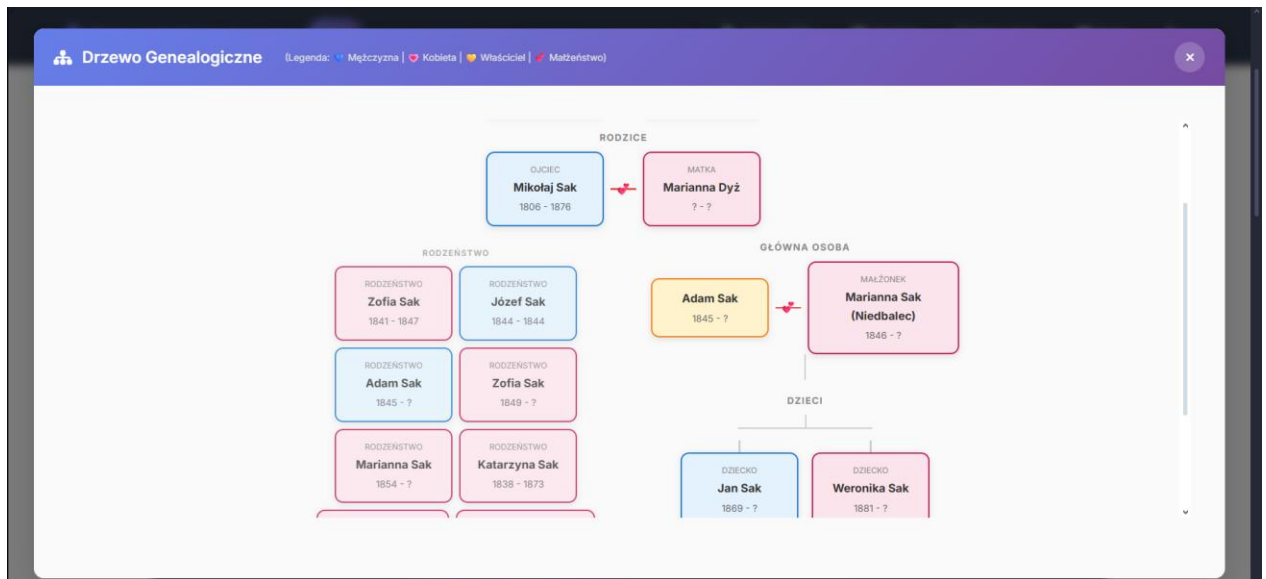
Drzewo genealogiczne (rys. 7.11) to bardziej szczegółowa i ustrukturyzowana wizualizacja, która koncentruje się na relacjach stricte rodzinnych. Została ona zaimplementowana od podstaw

z użyciem biblioteki **D3.js**, co pozwoliło na pełną kontrolę nad algorytmem pozycjonowania węzłów i renderowaniem grafiki SVG.

W jej skład wchodzi:

- **zaawansowany algorytm układu:**
 - **hierarchia pokoleń** – w przeciwieństwie do grafu powiązań, drzewo genealogiczne jest układem hierarchicznym, a zaimplementowany algorytm automatycznie analizuje relacje rodzic–dziecko i rozmieszcza osoby na odpowiednich poziomach pokoleniowych;
 - **grupowanie rodzin** – algorytm inteligentnie grupuje małżonków obok siebie na tym samym poziomie, a ich potomstwo umieszcza w pokoleniu poniżej, centralnie pod linią małżeństwa;
 - **dynamiczne obliczanie przestrzeni** – szerokość każdego węzła jest obliczana dynamicznie na podstawie długości tekstu (imienia i nazwiska), a odstępy między węzłami i pokoleniami są dostosowywane tak, aby uniknąć kolizji i zapewnić maksymalną czytelność;
 - **obsługa złożonych relacji** – algorytm został zaprojektowany tak, aby poprawnie radzić sobie ze złożonymi przypadkami, na przykład wizualizując tę samą osobę (córkę) zarówno w drzewie genealogicznym jej rodziców (ród pochodzenia), jak i w drzewie rodziny jej męża (ród poślubiony), co jest kluczowe dla odzwierciedlenia historycznych realiów;

- **interfejs i wizualizacja**
 - **dynamiczne renderowanie** – drzewo jest generowane na żądanie po kliknięciu odpowiedniego przycisku w panelu administratora lub na stronie protokołu i jest ono wyświetlane w dedykowanym oknie modalnym, co pozwala skupić się na analizie bez opuszczania bieżącego widoku;
 - **czytelna prezentacja** – każdy węzeł w drzewie zawiera kluczowe informacje: imię i nazwisko, lata życia oraz symbol płci, a jego kolor zależy od pokolenia, zaś osoba stanowiąca punkt wyjścia dla generowania drzewa jest specjalnie wyróżniona;
 - **pełna interaktywność** – użytkownik może przesuwając (pan) i powiększać (zoom) widok drzewa, co jest niezbędne przy analizie dużych, wielopokoleniowych rodzin, a podwójne kliknięcie na dowolną osobę w drzewie powoduje dynamiczne przerysowanie całej wizualizacji, ustawiając tę osobę jako nowy punkt centralny.



Rys. 7.11. Interaktywna wizualizacja drzewa genealogicznego wygenerowana w oknie modalnym

7.5. Strona wprowadzająca i materiały uzupełniające

W celu zapewnienia kompletności projektu oraz dostarczenia użytkownikowi pełnego kontekstu merytorycznego, utworzono dwa kluczowe komponenty webowe: profesjonalną stronę główną oraz podstronę z rysem historycznym.

7.5.1. Strona główna (strona_glowna/index.html)

Strona główna, widoczna na rysunku 7.12, pełni rolę profesjonalnej wizytówki i centralnego punktu wejścia do całego ekosystemu aplikacji. Została zaprojektowana jako nowoczesny “landing page”, którego celem jest nie tylko informowanie, ale również budowanie atmosfery i zanurzenie użytkownika w historycznym kontekście projektu od pierwszej chwili.

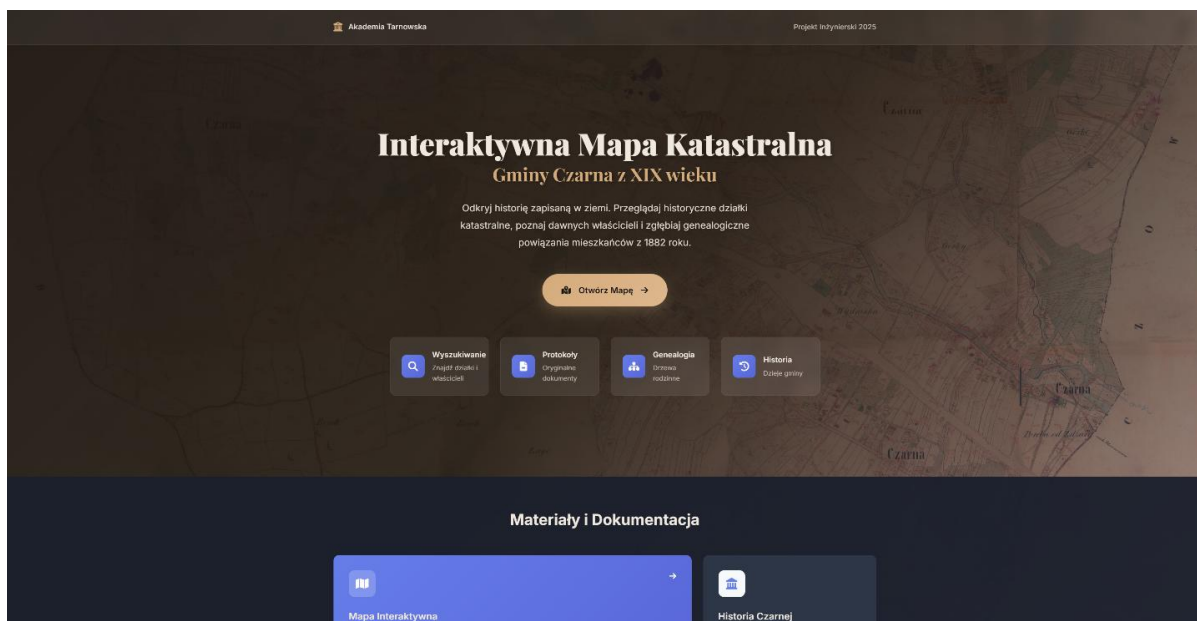
Projekt wizualny i atmosfera – interfejs strony został starannie zaprojektowany z dbałością o estetykę. Wykorzystuje wysokiej jakości typografię (m.in. szeryfową czcionkę *Playfair Display* dla nagłówków), subtelne animacje pojawiania się elementów oraz, co najważniejsze, **tło z nałożoną teksturą historycznej mapy katastralnej**. Całość pokryta jest gradientem, który zapewnia czytelność tekstu, jednocześnie nie tracąc historycznego charakteru. Takie podejście ma na celu wzbudzenie zainteresowania i podkreślenie archiwalnego charakteru danych.

Centralny hub nawigacyjny – strona nie jest jedynie statyczną wizytówką. Stanowi ona funkcjonalne centrum nawigacyjne, z którego użytkownik może jednym kliknięciem przejść do wszystkich kluczowych modułów systemu:

- Interaktywnej Mapy;
- Panelu Statystyk;
- Grafu Powiązań;
- Modułu Genealogii;
- Dokumentacji Technicznej;

A także może pobrać kartę projektu pracy inżynierskiej.

Struktura i treść – strona w zwięzły i przystępny sposób komunikuje cel i zakres projektu, prezentuje jego najważniejsze funkcjonalności w formie estetycznych kart, a także zawiera formalne informacje o autorze i opiece naukowej pracy, co podkreśla jej akademicki charakter.



Rys. 7.12.: Strona główna projektu pełniąca rolę centrum nawigacyjnego

7.5.2. Rys historyczny Gminy Czarna (strona_glowna/historia.html)

Aby dostarczyć użytkownikom niezbędnego kontekstu merytorycznego, przygotowano dedykowaną podstronę zawierającą krótki rys historyczny Gminy Czarna, który przedstawiono na rysunku 7.13. Nie jest to jedynie dodatek, lecz integralna część projektu, która wzbogaca wartość edukacyjną i naukową całego przedsięwzięcia.

Treść merytoryczna – strona prezentuje kluczowe informacje historyczne dotyczące powstania i rozwoju miejscowości, jej uwarunkowań geograficznych i gospodarczych, a także wydarzeń kluczowych dla okresu, z którego pochodzą dane katastralne, takich jak budowa kolei żelaznej. Informacje te pozwalają użytkownikowi lepiej zrozumieć tło społeczne i ekonomiczne, które ukształtowało strukturę własnościową widoczną na mapie.

Integracja z materiałem źródłowym – tekst jest ilustrowany zdigitalizowanymi materiałami archiwalnymi, takimi jak historyczne zdjęcia (np. dworca kolejowego) oraz fragmenty oryginalnych protokołów. Pozwala to na bezpośrednie zapoznanie się z charakterem źródeł, na których opiera się cały projekt.

Spójność wizualna – podstrona historyczna utrzymuje spójność wizualną ze stroną główną, wykorzystując ten sam motyw graficzny z mapą w tle, co zapewnia płynne i jednolite doświadczenie użytkownika podczas nawigacji po całym serwisie.



Rys. 7.13. Podstrona z rysem historycznym gminy i galerią materiałów archiwalnych

7.5.3. Dokumentacja techniczna (docs/)

W celu zapewnienia łatwości wdrożenia i dalszego rozwoju projektu, przygotowano w formie responsywnej strony internetowej kompletną dokumentację techniczną (rys. 7.14). Zastępuje ona tradycyjny, statyczny dokument, oferując znacznie bardziej interaktywne i przystępne doświadczenie.

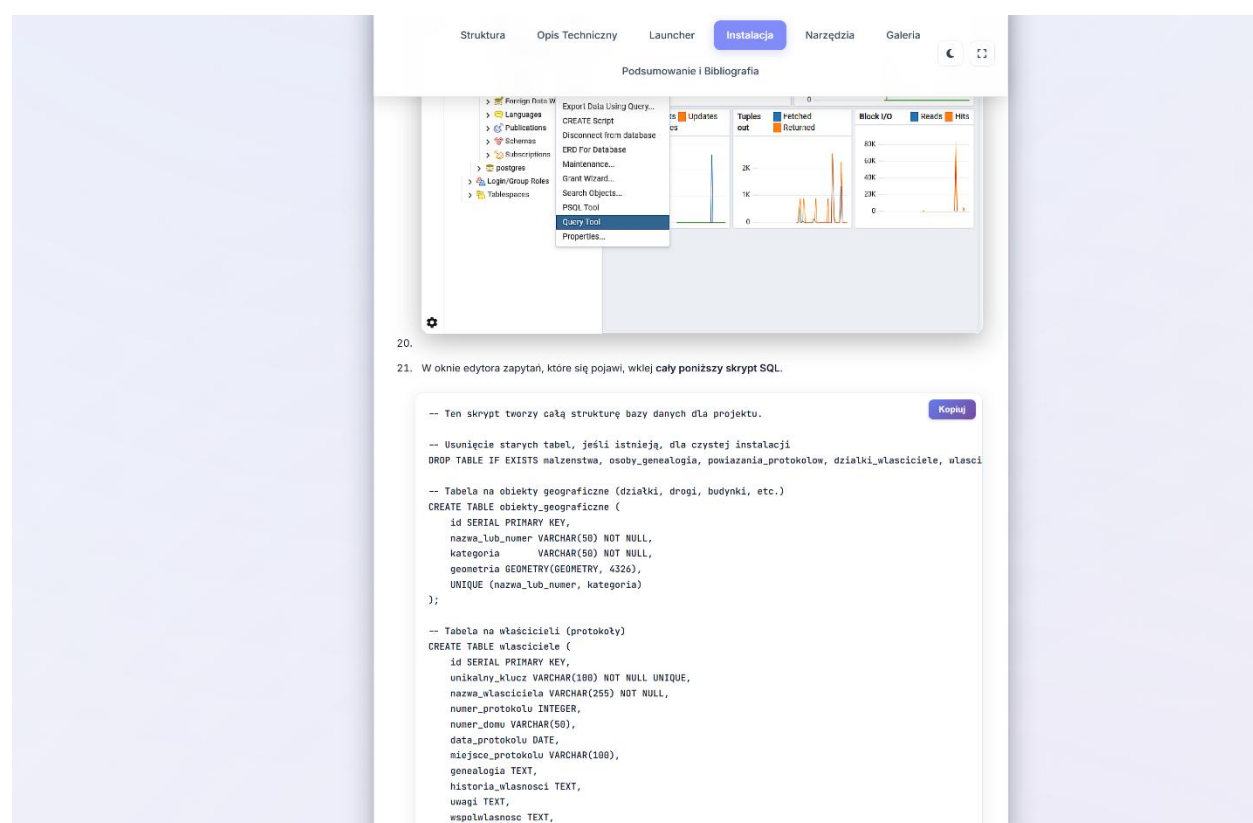
Struktura i nawigacja – dokumentacja została podzielona na logiczne rozdziały, które odpowiadają strukturze niniejszej pracy inżynierskiej. Posiada przyklejoną do górnej krawędzi

nawigację, która podświetla aktualnie przeglądaną sekcję, co znacząco ułatwia poruszanie się po obszernym materiale.

Interaktywne elementy – strona zawiera interaktywne bloki kodu z przyciskami “Kopiuj”, które ułatwiają wykonywanie komend instalacyjnych i konfiguracyjnych.

Przewodnik wdrożeniowy – kluczowym elementem dokumentacji jest szczegółowy przewodnik “krok po kroku”, który prowadzi nowego użytkownika przez cały proces – od instalacji wymaganego oprogramowania (Python, PostgreSQL), przez konfigurację bazy danych, aż po finalne uruchomienie aplikacji za pomocą Launchera.

Galeria aplikacji – w dokumentacji zintegrowano również galerię zrzutów ekranu, prezentującą wszystkie kluczowe widoki i narzędzia, co pozwala na szybkie zapoznanie się z możliwościami systemu.



Rys. 7.14. Widok interaktywnej dokumentacji technicznej projektu

Rozdział 8. Testowanie i zapewnienie jakości

W celu zapewnienia stabilności i poprawności działania logiki biznesowej, projekt został wyposażony w zestaw zautomatyzowanych testów jednostkowych i integracyjnych. Testy zostały napisane z użyciem popularnego frameworka pytest.

8.1. Cel i zakres testów

Głównym celem testów była weryfikacja poprawności działania endpointów *API* serwera backendowego. Zakres testów obejmował:

- **testy podstawowe** – sprawdzanie, czy kluczowe endpointy publiczne (np. */api/stats*) odpowiadają poprawnym kodem statusu (200 OK) i zwracają dane w oczekiwanej strukturze;
- **testy operacji *CRUD*** – weryfikacja pełnego cyklu życia danych (tworzenie, odczyt, aktualizacja, usuwanie) dla wszystkich modułów zarządzanych przez panel administratora;
- **testy mechanizmów bezpieczeństwa** – sprawdzanie czy panel administracyjny poprawnie zarządza sesją użytkownika (wymusza logowanie, gdy autoryzacja jest włączona, i zezwala na dostęp, gdy jest wyłączona);
- **testy przypadków brzegowych** – weryfikacja zachowania *API* w przypadku błędnych lub niekompletnych danych wejściowych;
- **redesign modułu testowego ("centrum testów")** – centralny interfejs graficzny wbudowany w Launcher, umożliwiający selektywne uruchamianie 13 kategorii weryfikacji oraz monitorowanie logów w czasie rzeczywistym z opcją eksportu do plików .log;
- **zaawansowane moduły weryfikacji danych ("guardian tests"):**
 - **moduł anty-duplikat** – wykorzystuje algorytmy podobieństwa ciągów znaków do analizy nazwisk, dat i numerów domów w celu wykrywania powielonych osób w

bazie genealogicznej;

– **weryfikacja przestrzenna sql** – wykorzystuje funkcje silnika PostGIS do automatycznego wykrywania błędów topologicznych, takich jak nakładanie się poligonów działek czy geometrie o zerowej powierzchni;

– **crawler zasobów** – sprawdza spójność referencyjną między bazą danych a systemem plików, weryfikując istnienie skanów protokołów i zdjęć powiązanych z rekordami;

– **detektor krzak-detector** – specjalistyczne narzędzie monitorujące poprawność kodowania UTF-8 w plikach danych, co eliminuje ryzyko wystąpienia błędów w wyświetlaniu polskich znaków diakrytycznych.

8.2. Środowisko testowe

Testy były uruchamiane w izolowanym środowisku, które nie narusza produkcyjnej bazy danych. Dzięki mechanizmowi “monkeypatching” biblioteki pytest, połączenia z bazą danych są przechwytywane i przekierowywane do tymczasowej, symulowanej bazy danych w pamięci, która jest tworzona na nowo przed każdym uruchomieniem testów.

8.3. Opis przypadków testowych

W tym podrozdziale zaprezentowano szczegółowy opis środowiska testowego oraz każdego przypadku testowego zaimplementowanego w katalogu backend/tests.

8.3.1. Fixtury i środowisko (conftest.py)

Mock bazy danych w pamięci – zaimplementowano klasę `_MemDB`, która symuluje działanie bazy danych, przechowując dane w listach i słownikach. Klasy `_FakeCursor` i `_FakeConn` przechwytyją zapytania SQL i wykonują operacje na tej strukturze w pamięci;

Monkeypatching – za pomocą fixtur pytest, funkcje łączące się z prawdziwą bazą danych (`get_db_connection`) są dynamicznie podmieniane na te, które korzystają z bazy w pamięci;

Dane inicjalne (*Seed*) – przed każdym testem baza w pamięci jest wypełniana minimalnym zestawem danych (np. 2 właścicieli, 5 obiektów), co zapewnia powtarzalność i izolację testów.

8.3.2. Testy podstawowe i bezpieczeństwa (test_basic.py, test_admin.py)

Testy podstawowe i testy bezpieczeństwa sprawdzają poprawne przekierowanie z głównego adresu URL. W ich ramach weryfikowane jest działanie mechanizmu autoryzacji w panelu administratora, a także sprawdzane jest, czy system poprawnie wymusza logowanie, gdy jest ono włączone i zezwala na dostęp, gdy jest wyłączone.

8.3.3. Testy operacji CRUD (test_api_crud.py, test_api_demografia.py, itp.)

Testy operacji CRUD implementują pełny cykl życia dla kluczowych obiektów (właściciel, wpis demograficzny). Scenariusz testowy obejmuje: utworzenie nowego obiektu (*POST*), odczytanie go (*GET*), zaktualizowanie (*PUT*), a na końcu usunięcie (*DELETE*) i weryfikację, sprawdzającą czy zasób faktycznie zniknął (oczekiwany kod 404).

8.3.4. Testy statystyk i błędów (test_stats.py, test_errors.py)

Testy statystyk i błędów weryfikują, czy *endpoint* /api/stats zwraca dane w poprawnej strukturze i czy wartości zagregowane zgadzają się z danymi inicjalnymi. Ponadto sprawdzają, czy próba dostępu do nieistniejącego zasobu zwraca poprawny kod błędu 404 Not Found.

8.4. Uruchamianie testów i dalszy rozwój

Testy można w prosty sposób uruchomić za pomocą dedykowanego przycisku w Launcherze lub ręcznie, za pomocą komendy w terminalu:

```
pytest backend/tests
```

Wynik jest prezentowany w zwężłej formie. Zalecane jest uruchamianie testów po każdej istotnej zmianie w logice *API*. W ramach dalszego rozwoju, zestaw testów można rozbudować o raporty pokrycia kodu (*coverage*) oraz testy wydajnościowe kluczowych endpointów.

Rozdział 9. Podsumowanie

Niniejsza inżynierska praca dyplomowa przedstawia kompleksowy proces tworzenia zaawansowanego systemu informatycznego zarządzającego i wizualizującego historyczne dane katastralne. Projekt, wykraczając poza prostą digitalizację, dostarczył w pełni funkcjonalny, interaktywny ekosystem, który skutecznie “ożywia” dane archiwalne, czyniąc je dostępnymi i użytecznymi dla szerokiego grona odbiorców.

9.1. Osiągnięte rezultaty

W ramach zrealizowanych prac osiągnięto wszystkie założone cele, zarówno na płaszczyźnie merytorycznej, jak i technologicznej. A najważniejsze rezultaty, które osiągnięto w pracy to:

1. Zintegrowany system Full–Stack

Zbudowano spójną aplikację opartą na nowoczesnych, otwartych technologiach (Python/Flask, PostgreSQL/PostGIS, JavaScript/Leaflet.js), która realizuje wszystkie założone cele. Architektura trójwarstwowa zapewniła modularność i skalowalność rozwiązania.

2. Skuteczna procedura digitalizacji i udostępniania danych

Historyczne, papierowe dane z Archiwum Państwowego i Diecezjalnego w Tarnowie zostały przetworzone na dynamiczną, cyfrową bazę wiedzy. Aplikacja webowa umożliwia intuicyjną eksplorację tych danych, demokratyzując dostęp do cennego dziedzictwa historycznego.

3. Zaawansowane funkcjonalności analityczne

Poza podstawową wizualizacją na mapie, system został wyposażony w bogaty zestaw modułów analitycznych, w tym *dashboard* ze statystykami, interaktywne rankingi właścicieli, porównywarke protokołów oraz innowacyjne narzędzia do wizualizacji sieci społecznych (graf powiązań i drzewa genealogiczne).

4. Profesjonalny ekosystem narzędziowy

Opracowano i zaimplementowano dedykowane, graficzne narzędzia deweloperskie (Launcher, Edytor Właścicieli, Edytor Działek, Edytor Genealogii), które automatyzują i zabezpieczają proces wprowadzania danych. Świadczy to o dojrzałym podejściu inżynierskim, wykraczającym poza samą implementację aplikacji końcowej i skupiającym się na całym cyklu życia danych.

5. System proaktywnego monitorowania spójności

Wdrożenie autorskiego mechanizmu *Background Guardian* wraz z algorytmami anty-duplikat i weryfikacją przestrzenną SQL pozwoliło na zautomatyzowanie nadzoru nad jakością danych. Przy skali projektu przekraczającej tysiące rekordów, automatyzacja ta stała się niezbędnym elementem inżynierskim, gwarantującym spójność materiałów historycznych, której nie dałoby się zapewnić drogą manualnej kontroli.

6. Uniwersalny silnik "*Multi-location*"

Jednym z najważniejszych osiągnięć projektu jest wyjście poza ramy dedykowanego rozwiązania dla jednej wsi. Zaimplementowano pełną obsługę wielomiejscowości, gdzie dodanie nowej gminy sprowadza się do utworzenia folderu z konfiguracją i zaimportowania danych. Aplikacja automatycznie zarządza wieloma bazami danych i dynamicznie dostosowuje interfejs, co czyni system gotowym produktem do wdrażania w innych jednostkach terytorialnych.

9.2. Wnioski końcowe

Realizacja projektu potwierdziła, że zastosowanie nowoczesnych technologii webowych i bazodanowych jest wysoce efektywną metodą na zabezpieczenie, analizę i popularyzację danych archiwalnych. Zaimplementowany system nie tylko rozwiązuje problem niedostępności i podatności na zniszczenie historycznych dokumentów, ale również otwiera nowe możliwości badawcze, pozwalając na odkrywanie złożonych relacji społeczno–gospodarczych, które byłyby niemożliwe do zidentyfikowania przy użyciu tradycyjnych metod.

Największym wyzwaniem technicznym okazała się integracja różnorodnych danych – opisowych, przestrzennych i genealogicznych – w jednym, spójnym modelu. Udało się to osiągnąć dzięki starannemu projektowi bazy danych oraz implementacji dedykowanych narzędzi, które zapewniły integralność danych na każdym etapie ich przetwarzania.

Realizacja tego projektu była dla mnie niezwykle cennym doświadczeniem, które pozwoliło mi zmierzyć się z szerokim spektrum wyzwań, zarówno na płaszczyźnie technicznej, jak i merytorycznej. Największą satysfakcję, ale i techniczną trudność, stanowiło dla mnie stworzenie interaktywnego drzewa genealogicznego. Zaprojektowanie algorytmu, który poprawnie modeluje

i wizualizuje złożone, wielopokoleniowe relacje rodzinne, było zadaniem wymagającym, lecz jego pomyślne ukończenie przyniosło mi ogromną dumę.

Projekt nauczył mnie również pokory wobec danych historycznych. Żmudny proces przepisywania protokołów z papieru, odcyfrowywanie trudnej do odczytania kaligrafii i domyślanie się znaczenia łacińskich zapisów były momentami frustrujące, ale uświadomiły mi, jak ważna jest precyzja i cierpliwość w pracy z materiałem archiwalnym. Równie ważną lekcją była dbałość o jakość kodu. Proces jego porządkowania i szczegółowego komentowania, choć początkowo wydawał się dodatkowym obowiązkiem, ostatecznie okazał się kluczowy dla zrozumienia własnej pracy i zapewnienia, że system będzie możliwy do utrzymania w przyszłości.

Projekt utwierdził mnie w przekonaniu, że najtrudniejsze decyzje, takie jak wybór odpowiedniej architektury i technologii na samym początku, mają największy wpływ na sukces całego przedsięwzięcia. Patrząc wstecz, mimo licznych trudności, stworzenie działającego narzędzia, które może służyć lokalnej społeczności, przyniosło mi ogromną satysfakcję. To doświadczenie ugruntowało moją pasję do tworzenia oprogramowania, które rozwiązuje realne problemy.

Ostatecznie, projekt dowodzi, że nawet z pozoru “suche” dane katastralne mogą stać się podstawą do zbudowania fascynującej, interaktywnej opowieści o historii lokalnej społeczności.

9.3. Propozycje dalszego rozwoju

Zbudowana architektura systemu jest modularna i elastyczna, co otwiera szerokie możliwości dalszego jego rozwoju. Poniżej przedstawiono kilka potencjalnych kierunków:

1. Rozbudowa modułu genealogicznego

Obecny moduł genealogiczny można rozbudować o bardziej zaawansowane funkcje, takie jak automatyczne wykrywanie potencjalnych powiązań na podstawie nazwisk i dat, czy graficzną wizualizację szlaków migracji rodzin.

2. Porównywanie map w czasie

System można rozszerzyć o możliwość wgrywania i porównywania map katastralnych z różnych okresów historycznych. Umożliwiłoby to wizualną analizę zmian w strukturze własnościowej na przestrzeni dziesięcioleci.

3. Integracja z zewnętrznymi bazami danych

Aplikację można zintegrować z zewnętrznymi bazami danych genealogicznymi (np. przez *API*), co pozwoliłoby na automatyczne wzbogacanie danych o osobach.

Bibliografia

- [1] "Flask Documentation," Pallets Projects. [Online].
<https://flask.palletsprojects.com/> Dostęp:2025r
- [2] "Leaflet – a JavaScript library for interactive maps," Leaflet. [Online].
<https://leafletjs.com/> Dostęp:2025r
- [3] "PostgreSQL Documentation," PostgreSQL Global Development Group. [Online]. Dostępne:
<https://www.postgresql.org/docs/> Dostęp:2025r
- [4] Archiwum Państwowe w Tarnowie, "Protokoły dochodzeń miejscowych celem założenia księgi hipotecznej dla gminy katastralnej Czarna," 1882.
- [5] Archiwum Diecezjalne w Tarnowie, "Księgi metrykalne parafii na terenie gminy Czarna," XIX wiek. Dostęp:2025r
-